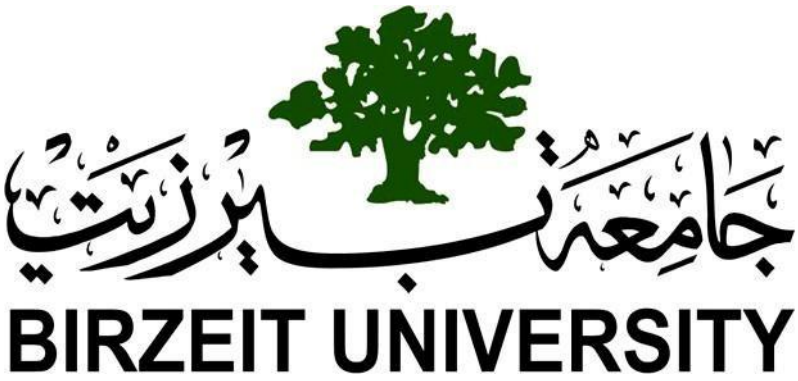


Faculty of Engineering & Technology

Electrical & Computer Engineering Department

COMPUTER ARCHITECTURE– ENCS4370

Project2

Design and Implementation of a 32-bit Multi-Cycle RISC Processor

Prepared by:

Aleen Othman – 1231626

Rahaf Alqam– 1232633

Waad Abu Rukab– 1230742

Section number: 1

Instructor: Dr. Ahmad Afaneh

Date: 17/09/2026

Table of Contents

Part 1 – Register File, ALU and Immediate Extender 5
1. Register File 5
2. Arithmetic Logic Unit (ALU) 6
3. Immediate Extender 7
4. Unit Verification 8
5. Assembly Test Program (test_student1.asm) 10
6. Instructions Verified by This Part 12

Part 2 – Memory, Program Counter and Temporary Registers 13
1. Instruction Memory 13
2. Data Memory 15
3. Program Counter and Next-PC Logic 17
4. Temporary Registers 19
5. Unit Verification 21
6. Verification on the Integrated Processor 24
7. Instructions Verified by This Part 25
8. Datapath Diagram 26

Part 3 – Control Unit, Integration and Verification 27
1. Instruction Execution and Multi-Cycle State Sequences 27
2. Control Unit and Control Signals 28
3. FSM Diagram and Boolean Control Logic 34
4. Top-Level Processor Integration 37
5. Verilog Implementation 40
6. Verification Methodology and Test Program 40
7. Final Verification Evidence 44
8. Design Assumptions and Verification Limitations 50
9. Conclusion 51
11. References 51

List of Figures

Figure 1 – Register file waveform. RegWrite pulses with Rd = 1, 2, F, 0, 3, 1; 8
Figure 2 – ALU waveform. A new operation every 10 ns; Result changes in the same instant as the inputs (combinational). Zero is high only where Result = 0. 9
Figure 3 – Immediate extender waveform. The same input 3FFFB produces FFFFFFFB while ExtOp = 1 and 0003FFFB while ExtOp = 0. 10
Figure 4 – Processor waveform while executing test_student1.hex (PC, IR, state, RegWrite, WriteReg, WriteData). 12
Figure 5 – Instruction memory waveform. 21
Figure 6 – Data memory waveform. 21
Figure 7 – Program counter waveform. 22
Figure 8 – Next-PC logic waveform. 22
Figure 9 – Temporary registers waveform. 23
Figure 10 – datapath: Instruction Memory, Program Counter, Next-PC Logic, Data Memory, and Temporary Registers, with their connections to the Register File, ALU, and Control Unit. 26
Figure 11 – Multi-Cycle Processor Control FSM derived from control_unit.v. 34
Figure 12 – Control logic derived from the implemented FSM output equations. 36
Figure 13 – Main multi-cycle datapath integration and major control points. 38
Figure 14 – Program-counter and control-flow integration. 39
Figure 15 – ADDI write-back sequence for ADDI R1, R0, 5. 45
Figure 16 – SLL execution with shift source R9 = 0x00000021 (33). 46
Figure 17 – SRL execution with the same masked shift amount. 46
Figure 18 – SW memory-write cycle for SW R3, 0(R0). 47
Figure 19 – LW memory-read and register write-back sequence for LW R13, 0(R0). 47
Figure 20 – BEQ taken when the two operands are equal. 48
Figure 21 – BNE not taken for equal operands. 48
Figure 22 – BNE taken for unequal operands. 48
Figure 23 – JAL simultaneously updates the PC and writes the return address to R14. 49
Figure 24 – JR returns to the link address previously created by JAL. 49
Figure 25 – Lb sign-extension and write-back for Lb R13, 1(R0). 49

List of Tables

Table 1 – Register file interface 5
Table 2 – ALU interface 6
Table 3 – ALU operation encoding and the instructions that use each operation 6
Table 4 – Immediate extender interface 7
Table 5 – ExtOp value per instruction 7
Table 6 – Register file testbench cases and expected results 8
Table 7 – ALU testbench cases and expected results 9
Table 8 – Immediate extender testbench cases and expected results 10
Table 9 – Part 1 assembly test program (test_student1.asm) and its machine code 10
Table 10 – Expected register contents after executing test_student1.hex 11
Table 11 – Instruction memory interface 13
Table 12 – Data memory interface 15
Table 13 – Program counter interface 17
Table 14 – Next-PC logic interface 17
Table 15 – Temporary registers interface 19
Table 16 – Part 2 expected versus observed results on the integrated processor 24
Table 17 – FSM State Definitions 27
Table 18 – Instruction State Sequences and Cycle Counts 28
Table 19 – Control Signal Definitions and Activation Conditions 29
Table 20 – Encoded Selector Control Signals 30
Table 21 – ALUOp Encoding Used by the Control Unit 30
Table 22 – Functional Control Truth Table — Datapath, ALU, Memory, and PC Controls 31
Table 23 – Functional Control Truth Table — Temporary-Register and Destination Controls 32
Table 24 – Expansion of Opcode- and Condition-Dependent Truth-Table Entries 33
Table 25 – Integrated modules used by top_processor.v 37
Table 26 – Final integration simulation environment 40
Table 27 – Machine-code program loaded into Instruction Memory by final_processor_tb.v 42
Table 28 – Expected versus observed final architectural state from the completed XSim regression 44
Table 29 – Verified instruction set 44

Part 1 – Register File, ALU and Immediate Extender

This part covers the three datapath components designed by Student 1: the register file, the arithmetic logic unit (ALU) and the immediate extender. Each component was implemented in Verilog, verified with its own self-checking testbench, and then integrated into the multi-cycle processor. The section closes with the assembly test program written for the instructions that exercise these components, its machine code, and the result of running it on the integrated processor.

1. Register File

1.1 Description

The register file holds the sixteen 32-bit general-purpose registers R0–R15. It provides two asynchronous read ports (Reg(Rs) and Reg(Rt)) and one synchronous write port. Register R0 is hardwired to zero: any read of R0 returns 0 and any write to R0 is silently ignored, as required by the processor specification.

Reads are combinational (no clock), so the values of Reg(Rs) and Reg(Rt) are available in the same cycle in which the decode state is entered and can be latched directly into the A and B temporary registers. Writes occur on the rising clock edge when RegWrite = 1 and Rd ≠ 0, which is the write-back state of the control FSM.

1.2 Interface

Table 1 – Register file interface

Port	Dir	Width	Description
clk	in	1	System clock
rst	in	1	Synchronous reset, clears all registers
RegWrite	in	1	Write enable (from control unit)
Rs	in	4	Read port 1 address
Rt	in	4	Read port 2 address
Rd	in	4	Write address
WriteData	in	32	Value written to Reg(Rd)
ReadData1	out	32	Reg(Rs), asynchronous
ReadData2	out	32	Reg(Rt), asynchronous

1.3 Implementation

```
reg [31:0] regs [0:15];

// asynchronous read, R0 always reads as zero
assign ReadData1 = (Rs == 4'd0) ? 32'd0 : regs[Rs];
assign ReadData2 = (Rt == 4'd0) ? 32'd0 : regs[Rt];

// synchronous write, writes to R0 are ignored
always @(posedge clk) begin
    if (rst) begin
        for (i = 0; i < 16; i = i + 1) regs[i] <= 32'd0;
    end
    else if (RegWrite && Rd != 4'd0)
        regs[Rd] <= WriteData;
end
```

1.4 Design decisions

- **Asynchronous read / synchronous write.** Reading does not modify state, so it is kept combinational to save one cycle in the ID state. Writing modifies state and is therefore clocked, so a write can only happen at a well-defined point (the WB state).
- **R0 protected on both sides.** The read mux returns 0 for address 0 regardless of the stored value, and the write condition excludes Rd = 0. Either mechanism alone would be sufficient; using both makes the behaviour independent of the memory's initial contents.
- **Reset input.** Not required by the specification, but added so that simulation starts from a known all-zero state instead of X values.

2. Arithmetic Logic Unit (ALU)

2.1 Description

The ALU is a purely combinational unit that takes two 32-bit operands A and B and a 4-bit operation select ALUOp, and produces a 32-bit Result together with a Zero flag. Operand A is always Reg(Rs). Operand B is either Reg(Rt) or the extended immediate, selected by the ALUSrc multiplexer outside the ALU. The Zero flag is asserted when Result = 0 and is used by the control unit to resolve BEQ/BNE after a SUB operation.

2.2 Interface

Table 2 – ALU interface

Port	Dir	Width	Description
A	in	32	Operand 1 = Reg(Rs)
B	in	32	Operand 2 = Reg(Rt) or ExtImm
ALUOp	in	4	Operation select
Result	out	32	Operation result
Zero	out	1	1 when Result == 0

2.3 Operation encoding

ALUOp uses the same numeric values as the R-type opcodes in the ISA. This means the control unit can pass opcode[3:0] directly for R-type instructions without a translation table.

Table 3 – ALU operation encoding and the instructions that use each operation

ALUOp	Operation	Result	Used by
0000	ADD	A + B	ADD, ADDI, LW, Lb, SW (address)
0001	SUB	A − B	SUB, BEQ, BNE (compare)
0010	AND	A & B	AND, ANDI
0011	OR	A | B	OR, ORI
0100	XOR	A ^ B	XOR
0101	NOR	~(A | B)	NOR
0110	SLL	A << B[4:0]	SLL
0111	SRL	A >> B[4:0]	SRL
1000	CLR	0	CLR

2.4 Implementation

```
always @(*) begin
    case (ALUOp)
        4'b0000: Result = A + B;        // ADD
        4'b0001: Result = A - B;        // SUB
        4'b0010: Result = A & B;        // AND
        4'b0011: Result = A | B;        // OR
        4'b0100: Result = A ^ B;        // XOR
        4'b0101: Result = ~(A | B);     // NOR
        4'b0110: Result = A << B[4:0];  // SLL
        4'b0111: Result = A >> B[4:0];  // SRL
        4'b1000: Result = 32'd0;        // CLR
        default: Result = 32'd0;
    endcase
end
assign Zero = (Result == 32'd0);
```

2.5 Design decisions

- **Combinational design.** The ALU stores nothing; the result is latched into the ALUOut temporary register by the control unit, keeping the ALU itself stateless and reusable in every execute-type state.
- **Shift amount uses B[4:0] only.** Operands are 32 bits wide, so valid shift amounts are 0–31. Only the five least significant bits of Reg(Rt) are used; higher bits are ignored (e.g. a shift amount of 33 behaves as 1).
- **SRL is a logical shift.** Vacated bits are filled with zeros, not with the sign bit.
- **Two's complement arithmetic.** ADD and SUB operate on the raw 32-bit patterns, so negative values in two's complement are handled without any special logic.
- **Zero flag instead of a separate comparator.** BEQ/BNE are resolved by performing SUB and checking Zero, which avoids a dedicated 32-bit comparator in the datapath.
- **CLR inside the ALU.** Implemented as an ALU operation so that CLR follows the same R-type state sequence as every other R-type instruction.

3. Immediate Extender

3.1 Description

I-type instructions carry an 18-bit immediate in IR[17:0]. The extender widens it to 32 bits so it can be used as an ALU operand or a branch offset. The specification requires sign extension for arithmetic, memory and branch instructions and zero extension for logical instructions; the choice is made by the control signal ExtOp.

3.2 Interface

Table 4 – Immediate extender interface

Port	Dir	Width	Description
`Imm`	in	18	Immediate field, IR[17:0]
`ExtOp`	in	1	1 = sign extend, 0 = zero extend
`ExtImm`	out	32	Extended immediate

Table 5 – ExtOp value per instruction

Instruction	ExtOp	Extension
`ADDI, LW, Lb, SW, BEQ, BNE`	`1`	Sign (copy Imm[17] into bits 31–18)
`ANDI, ORI`	`0`	Zero (bits 31–18 = 0)

3.3 Implementation

```
assign ExtImm = ExtOp ? {{14{Imm[17]}}, Imm}   // sign extension
                      : {14'b0, Imm};          // zero extension
```

3.4 Design decision

The extender does not decode the opcode itself; it receives ExtOp from the control unit. This keeps all instruction-decoding logic in one place (the control unit) and makes the extender a trivial, easily verified multiplexer.

4. Unit Verification

Each component has a self-checking testbench that drives inputs, compares outputs against expected values, and prints PASS or FAIL for every case. All testbenches were run in Active-HDL.

4.1 Register file testbench (register_file_tb.v)

Table 6 – Register file testbench cases and expected results

#	Test	Expected
1	Reset, then read R1 and R15	Both read 00000000
2	Write A→R1, FFFFFFF6→R2, DEADBEEF→R15; read back from both ports	Values read back exactly
3	Write 12345678 to R0 with RegWrite = 1	R0 still reads 00000000
4	Present data for R3 with RegWrite = 0	R3 remains 00000000
5	Overwrite R1 with 00000001	R1 = 1, R2 unchanged

Simulation result: all 6 checks passed.

```
PASS  R1=00000000  R15=00000000
PASS  R1=0000000a  R2=fffffff6
PASS  R15=deadbeef  R1=0000000a
PASS  R0=00000000  R0=00000000
PASS  R3=00000000  R0=00000000
PASS  R1=00000001  R2=fffffff6
Register file test done.
```

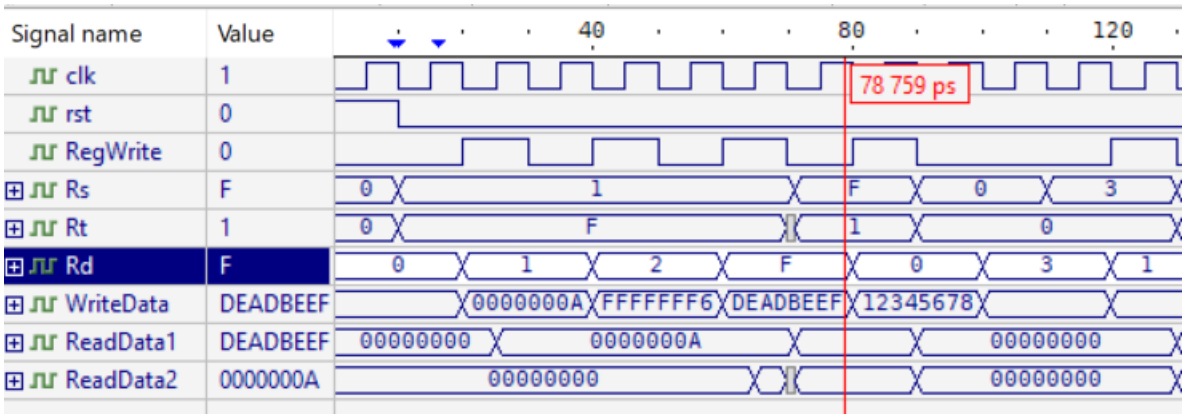


Figure 1 – Register file waveform. RegWrite pulses with Rd = 1, 2, F, 0, 3, 1; the write with Rd = 0 leaves ReadData1 at 0, proving R0 protection. ReadData changes immediately when Rs/Rt change (asynchronous read).

4.2 ALU testbench (alu_tb.v)

Fourteen operations are checked. Each check compares Result with the expected value and Zero with (expected == 0).

Table 7 – ALU testbench cases and expected results

Op	A	B	Expected	Purpose
ADD	00000005	00000007	0000000C	Basic add
ADD	0000000A	FFFFFFF6	00000000	10 + (−10) = 0, Zero = 1
SUB	00000007	00000005	00000002	Basic subtract
SUB	00000005	00000007	FFFFFFFE	Negative result (−2)
SUB	00000009	00000009	00000000	Equal operands, Zero = 1 (BEQ)
AND	FF00FF00	0FF00FF0	0F000F00	Bitwise AND
OR	FF00FF00	0FF00FF0	FFF0FFF0	Bitwise OR
XOR	FF00FF00	0FF00FF0	F0F0F0F0	Bitwise XOR
NOR	FF00FF00	0FF00FF0	000F000F	Bitwise NOR
SLL	00000001	00000004	00000010	Shift left by 4
SRL	80000000	0000001F	00000001	Shift right by 31
SLL	00000001	00000021	00000002	Amount 33 → uses [4:0] = 1
SRL	FFFFFFFF	00000004	0FFFFFFF	Logical shift, zero fill
CLR	DEADBEEF	FFFFFFFF	00000000	Clear, Zero = 1

Simulation result: all 14 checks passed.

```
PASS  ADD   A=00000005 B=00000007 -> 0000000c  Zero=0
PASS  ADD   A=0000000a B=fffffff6 -> 00000000  Zero=1
PASS  SUB   A=00000007 B=00000005 -> 00000002  Zero=0
PASS  SUB   A=00000005 B=00000007 -> fffffffe  Zero=0
PASS  SUB   A=00000009 B=00000009 -> 00000000  Zero=1
PASS  AND   A=ff00ff00 B=0ff00ff0 -> 0f000f00  Zero=0
PASS  OR    A=ff00ff00 B=0ff00ff0 -> fff0fff0  Zero=0
PASS  XOR   A=ff00ff00 B=0ff00ff0 -> f0f0f0f0  Zero=0
PASS  NOR   A=ff00ff00 B=0ff00ff0 -> 000f000f  Zero=0
PASS  SLL   A=00000001 B=00000004 -> 00000010  Zero=0
PASS  SRL   A=80000000 B=0000001f -> 00000001  Zero=0
PASS  SLL   A=00000001 B=00000021 -> 00000002  Zero=0
PASS  SRL   A=ffffffff B=00000004 -> 0fffffff  Zero=0
PASS  CLR   A=deadbeef B=ffffffff -> 00000000  Zero=1
ALU test done.
```

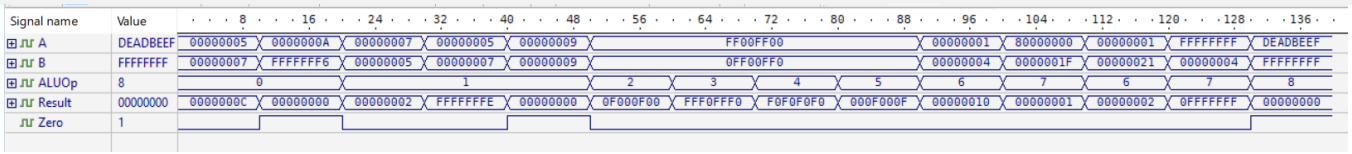


Figure 2 – ALU waveform. A new operation every 10 ns; Result changes in the same instant as the inputs (combinational). Zero is high only where Result = 0.

4.3 Immediate extender testbench (imm_extender_tb.v)

Table 8 – Immediate extender testbench cases and expected results

Imm (18-bit)	ExtOp	Expected	Purpose
00005	1	00000005	Positive value unchanged
3FFFB	1	FFFFFFFB	−5 stays −5
20000	1	FFFE0000	Most negative 18-bit value
1FFFF	1	0001FFFF	Most positive 18-bit value
00005	0	00000005	Zero extension
3FFFB	0	0003FFFB	Same bits as row 2, no sign fill
20000	0	00020000	Bit 17 set, zero extended

Simulation result: all 7 checks passed.

```
PASS  SIGN+  Imm=00005 ExtOp=1 -> 00000005
PASS  SIGN-  Imm=3fffb ExtOp=1 -> fffffffb
PASS  SIGN-  Imm=20000 ExtOp=1 -> fffe0000
PASS  SIGN+  Imm=1ffff ExtOp=1 -> 0001ffff
PASS  ZERO   Imm=00005 ExtOp=0 -> 00000005
PASS  ZERO   Imm=3fffb ExtOp=0 -> 0003fffb
PASS  ZERO   Imm=20000 ExtOp=0 -> 00020000
Immediate extender test done.
```

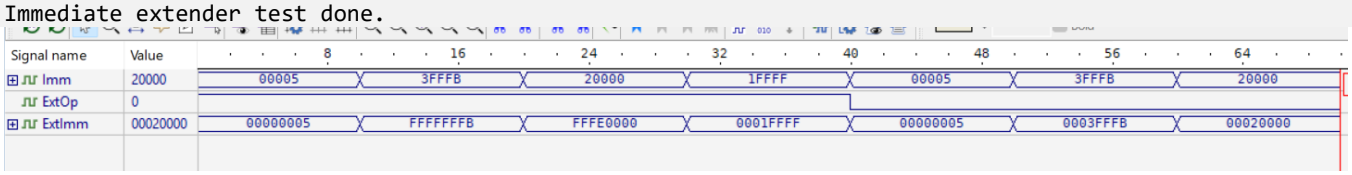


Figure 3 – Immediate extender waveform. The same input 3FFFB produces FFFFFFFB while ExtOp = 1 and 0003FFFB while ExtOp = 0.

5. Assembly Test Program (test_student1.asm)

5.1 Program

The program exercises all twelve instructions that depend on this part: the nine R-type ALU instructions and ADDI, ANDI, ORI. Each instruction uses results of the previous ones, so an error in any instruction propagates to a visible wrong register value.

Table 9 – Part 1 assembly test program (test_student1.asm) and its machine code

Addr	Instruction	Machine code	Effect
0	ADDI R1, R0, 5	28400005	R1 = 5
1	ADDI R2, R0, -3	2883FFFD	R2 = FFFFFFFD (sign extension)
2	ADD R3, R1, R2	00C48000	R3 = 2
3	SUB R4, R1, R2	05048000	R4 = 8
4	AND R5, R1, R4	09450000	R5 = 0
5	OR R6, R1, R4	0D850000	R6 = D
6	XOR R7, R1, R3	11C4C000	R7 = 7
7	NOR R8, R0, R1	16004000	R8 = FFFFFFFA
8	ADDI R9, R0, 2	2A400002	R9 = 2 (shift amount)

Addr	Instruction	Machine code	Effect
9	SLL R10, R1, R9	1A864000	R10 = 14 (5 << 2)
10	SRL R11, R4, R9	1ED24000	R11 = 2 (8 >> 2)
11	ANDI R12, R2, 0xFF	2F0800FF	R12 = FD (zero extension)
12	ORI R13, R0, 0x3FFFF	3343FFFF	R13 = 0003FFFF (zero, not sign)
13	CLR R3	20C00000	R3 = 0

5.2 Encoding example

SUB R4, R1, R2 (R-type: opcode6 | Rd4 | Rs4 | Rt4 | unused14):

```
opcode = 1  -> 000001
Rd = 4      -> 0100
Rs = 1      -> 0001
Rt = 2      -> 0010
unused      -> 00000000000000
000001 0100 0001 0010 00000000000000 = 0x05048000
```

ADDI R2, R0, -3 (I-type: opcode6 | Rt4 | Rs4 | Imm18):

```
opcode = 10 -> 001010
Rt = 2      -> 0010
Rs = 0      -> 0000
Imm = -3    -> 111111111111111101  (0x3FFFD, 18-bit two's complement)
001010 0010 0000 111111111111111101 = 0x2883FFFD
```

5.3 Expected register contents after execution

Table 10 – Expected register contents after executing test_student1.hex

Reg	Value	Reg	Value
R1	00000005	R8	FFFFFFFA
R2	FFFFFFFD	R9	00000002
R3	00000000	R10	00000014
R4	00000008	R11	00000002
R5	00000000	R12	000000FD
R6	0000000D	R13	0003FFFF
R7	00000007		

5.4 Result on the integrated processor

The machine code was loaded into the instruction memory of the complete multi-cycle processor (student1_program_tb.v) and executed. All thirteen registers matched the expected values.

```
PASS R1 = 00000005
PASS R2 = fffffffd
PASS R3 = 00000000
PASS R4 = 00000008
PASS R5 = 00000000
PASS R6 = 0000000d
PASS R7 = 00000007
```

```
PASS R8 = fffffffa
PASS R9 = 00000002
PASS R10 = 00000014
PASS R11 = 00000002
PASS R12 = 000000fd
PASS R13 = 0003ffff
STUDENT1 PROGRAM: PASS
```

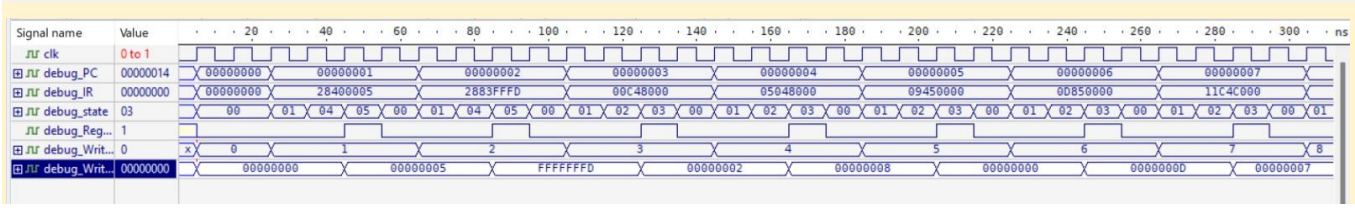


Figure 4 – Processor waveform while executing test_student1.hex (PC, IR, state, RegWrite, WriteReg, WriteData).

6. Instructions Verified by This Part

ADD, SUB, AND, OR, XOR, NOR, SLL, SRL, CLR, ADDI, ANDI, ORI — all verified both at unit level (ALU / extender testbenches) and at processor level (test_student1.hex on the integrated design).

Part 2 – Memory, Program Counter and Temporary Registers

This part covers the four state-holding datapath components designed by Partner 2 (Rahaf): the instruction memory, the data memory, the program counter together with its next-PC logic, and the temporary registers (IR, A, B, ALUOut, MDR, OldPC). Together these modules store the processor's architectural and micro-architectural state between the cycles of the multi-cycle FSM and expose the read/write points that the Control Unit (Partner 3) drives in every state.

1. Instruction Memory

1.1 Description

Instruction Memory stores the 32-bit machine-code program executed by the processor. It is implemented as a 256-word, word-addressed array, consistent with the project specification's requirement that PC advances by 1 per instruction rather than by 4. The read is purely combinational: the instruction word at the current address is available in the same cycle, which lets the IF state capture it into IR (IRWrite) without an extra wait cycle.

1.2 Interface

Table 11 – Instruction memory interface

Port	Dir	Width	Description
`address`	in	32	Word address of the instruction to fetch (typically the current PC). Only address[7:0] is decoded, giving 256 addressable words.
`instruction`	out	32	Instruction word stored at address, available combinationally (no clock).

1.3 Implementation

```
module instruction_memory(
    input [31:0] address,
    output [31:0] instruction
);

reg [31:0] memory [0:255];

assign instruction = memory[address[7:0]];

endmodule
```

1.4 Design decisions

- **Word-addressed, 256 entries.** The project specification uses PC = PC + 1 for sequential execution, which only produces the correct next instruction if Instruction Memory is indexed one word at a time rather than one byte at a time. A 256-entry array is large enough for every unit test program and for the 47-instruction shared integration program while keeping the address decode small (address[7:0]).
- **Combinational read, no write port.** The module is read-only from the processor's point of view: it has no clk, MemWrite, or WriteData input. This matches its role in the datapath, where only the Control Unit's IF state consumes its output; nothing in the specified instruction set writes to Instruction Memory at run time.
- **Program loaded by direct initialization, not $readmemh.** The current RTL preloads a short 3-word sample program in an initial block (memory[0]/[1]/[2]) rather than reading a hex file with $readmemh, and the standalone instruction_memory_tb.v exercises exactly that sample. For the shared final-integration run, Partner 3's final_processor_tb.v writes the full 47-word final_integration program directly into this same

memory array at simulation time 0 before releasing reset, so the integrated processor still executes the intended program end-to-end. Adding a general $readmemh-based loader to instruction_memory.v itself remains open; it is listed as a known limitation in Part 3, Section 8 ('Instruction Memory loading').

2. Data Memory

2.1 Description

Data Memory holds the operands read and written by LW, Lb, SW, and SWAP. Per the team's pre-coding design decisions (confirmed against the instructor's clarification), both reads and writes are word-addressed, and Lb reads the addressed 32-bit word and extracts its low byte, sign-extending it to 32 bits. Consistent with the project specification, reads are asynchronous (combinational, so ReadData tracks address immediately) and writes are synchronous (registered on the rising clock edge when MemWrite is asserted), so a value written in one cycle becomes visible to a read of the same address from the following cycle onward.

2.2 Interface

Table 12 – Data memory interface

Port	Dir	Width	Description
clk	in	1	Clock; writes are registered on its rising edge.
MemRead	in	1	Asserted by the Control Unit during a memory-read state. Present for interface symmetry with the control signals; see the design-decision note below.
MemWrite	in	1	When high on a rising clock edge, memory[address] is loaded with WriteData.
LB	in	1	Selects the Lb (load-byte) read path: when high, ReadData is the sign-extended low byte of the addressed word instead of the full word.
address	in	32	Word address applied to both the read and the write ports.
WriteData	in	32	Data written to memory[address] when MemWrite is asserted.
ReadData	out	32	word_read for LW/SWAP, or the sign-extended byte word_read[7:0] for Lb.

2.3 Implementation

```
module data_memory (
    input         clk,
    input         MemRead,
    input         MemWrite,
    input         LB,
    input  [31:0] address,
    input  [31:0] WriteData,
    output [31:0] ReadData
);

    parameter MEM_DEPTH = 256;
    reg [31:0] memory [0:MEM_DEPTH-1];

    // Synchronous write
    always @(posedge clk) begin
        if (MemWrite)
            memory[address] <= WriteData;
    end

    // Asynchronous read
    wire [31:0] word_read = memory[address];

    // LB sign extension
```

```
    wire [31:0] lb_extended =
        {{24{word_read[7]}}, word_read[7:0]};

    assign ReadData = LB ? lb_extended : word_read;

endmodule
```

2.4 Design decisions

- **Asynchronous read / synchronous write.** Reading does not modify state, so ReadData is kept combinational and is valid the moment address settles, matching the same asynchronous-read convention used by the Register File in Part 1. Writing modifies state, so it is registered on clk and only commits when MemWrite is explicitly asserted, giving the Control Unit a well-defined single-cycle write point (MEM_WRITE / SWAP_COMMIT).
- **MemRead does not gate the read path.** ReadData is driven by address unconditionally; MemRead is not used inside this module to enable or disable the read. The signal is still generated by the Control Unit as a read-state indicator (and drives MDRWrite elsewhere in the datapath), but the memory itself is read continuously. This is called out explicitly as an implementation note in Part 3, Section 8, so the behaviour is intentional and understood by the whole team rather than an oversight.
- **Lb: word read, byte 0 selected, sign-extended.** The team's design-decision log flagged Lb's exact extension (sign vs. zero) and byte-selection policy as an open question for the instructor before coding. The implemented and integrated behaviour — read the addressed word, take its low byte word_read[7:0], and sign-extend — was adopted as the working assumption and is the version exercised by every testbench and by the shared final-integration regression; Part 3, Section 8 records the same assumption as the current design basis.
- **256-word depth.** MEM_DEPTH = 256 keeps address as a direct array index without additional decoding logic. Every test program used by the team, including the 47-instruction shared regression, stays within this range; addresses at or beyond 256 are outside the modelled range in simulation, consistent with the limitation already noted for the data memory in Part 3, Section 8.

3. Program Counter and Next-PC Logic

3.1 Program Counter

The Program Counter is a simple 32-bit synchronous register that holds the address of the instruction currently being fetched. It loads nextPC (supplied by the Next-PC Logic module) only when PCWrite is asserted by the Control Unit, and resets to 0.

Table 13 – Program counter interface

Port	Dir	Width	Description
clk	in	1	Clock.
rst	in	1	Synchronous reset; forces PC to 0.
PCWrite	in	1	Write enable, asserted by the Control Unit in IF and in every control-flow state (BRANCH taken, J, JAL, JR).
nextPC	in	32	Value to load into PC, computed by the Next-PC Logic module.
PC	out	32	Current program counter value, fed to Instruction Memory and latched into OldPC during IF.

```
module program_counter(
    input clk,
    input rst,
    input PCWrite,
    input [31:0] nextPC,
    output reg [31:0] PC
);

always @(posedge clk) begin
    if (rst)
        PC <= 32'b0;
    else if (PCWrite)
        PC <= nextPC;
end

endmodule
```

3.2 Next-PC Logic

Next-PC Logic is a purely combinational multiplexer that selects the next value for the Program Counter according to the 2-bit PCSrc selector agreed with the Control Unit (Partner 3). All three arithmetic sources are computed from OldPC — the address of the instruction currently in flight, latched during IF — rather than from the (possibly already-advanced) PC register itself, per the team's pre-coding decision on what 'PC' means in the control-flow equations.

Table 14 – Next-PC logic interface

Port	Dir	Width	Description
OldPC	in	32	Address of the instruction currently executing, from the temporary-register file.
offset	in	32	Sign-extended immediate/offset supplied by the Immediate Extender (Part 1), reused for BEQ/BNE and for J/JAL.
rs_value	in	32	Reg(Rs), used only for JR.
PCSrc	in	2	Source select: 00 sequential, 01 branch, 10 jump/jump-and-link, 11 register jump.
nextPC	out	32	Value to be loaded into PC when PCWrite is asserted.

```
module next_pc_logic(
    input  [31:0] OldPC,
    input  [31:0] offset,
    input  [31:0] rs_value,
    input  [1:0] PCSrc,
    output reg [31:0] nextPC
);

always @(*) begin
    case(PCSrc)
        2'b00: begin
            // Normal execution
            nextPC = OldPC + 1;
        end
        2'b01: begin
            // BEQ / BNE
            nextPC = OldPC + offset;
        end
        2'b10: begin
            // J / JAL
            nextPC = OldPC + offset;
        end
        2'b11: begin
            // JR
            nextPC = rs_value;
        end
        default:
            nextPC = OldPC + 1;
    endcase
end

endmodule
```

3.3 Design decisions

- **All targets computed from OldPC, not PC.** The team agreed before coding that 'PC' in the specification's control-flow equations (PC = OldPC + sign_extend(Imm), PC = OldPC + sign_extend(offset), R14 = OldPC + 1) refers to the address of the instruction currently being executed, which is why that address is captured into OldPC during IF and every arithmetic nextPC source in this module reads OldPC rather than the live PC register.
- **BEQ/BNE and J/JAL share one adder path.** Both use OldPC + offset; only the width and sign-extension of offset differ upstream (18-bit I-type immediate vs. 26-bit J-type offset), so PCSrc = 01 and PCSrc = 10 are functionally identical inside this module and are only distinguished so the Control Unit can separately gate PCWrite on the branch condition for PCSrc = 01.
- **JR bypasses the adder entirely.** PCSrc = 11 routes rs_value (Reg(Rs), read from the Register File) straight to nextPC, matching PC = Reg(Rs) from the specification and letting JR return to a previously saved address such as the one written into R14 by JAL.
- **Default case.** The case statement's default falls back to OldPC + 1 (sequential execution) rather than leaving nextPC undefined, so an unreachable PCSrc encoding cannot propagate X values into the Program Counter during simulation.

4. Temporary Registers

4.1 Description

The temporary-register file holds the six pieces of micro-architectural state that let the multi-cycle datapath carry values from one clock cycle to the next: IR (fetched instruction), A and B (the two register-file operands read in ID), ALUOut (the ALU result, or a computed effective address), MDR (the value read back from Data Memory), and OldPC (the address of the instruction currently executing, captured during IF). Each register has its own write enable, so the Control Unit updates only the registers that are meaningful in the current state instead of writing all six every cycle.

4.2 Interface

Table 15 – Temporary registers interface

Port	Dir	Width	Description
`clk, rst`	in	1 each	Shared clock and synchronous reset; reset clears all six registers to zero.
`IRWrite / AWrite / BWrite / ALUOutWrite / MDRWrite / OldPCWrite`	in	1 each	Independent per-register write enables driven by the Control Unit.
`instruction, regA, regB, alu_result, mem_data, PC`	in	32 each	Data sources for IR, A, B, ALUOut, MDR, and OldPC respectively.
`IR, A, B, ALUOut, MDR, OldPC`	out	32 each	Registered outputs, read by the Register File address decode, the ALU, Data Memory, and Next-PC Logic.

4.3 Implementation

```
module temporary_registers(
    input clk,
    input rst,
    input IRWrite,
    input AWrite,
    input BWrite,
    input ALUOutWrite,
    input MDRWrite,
    input OldPCWrite,
    input [31:0] instruction,
    input [31:0] regA,
    input [31:0] regB,
    input [31:0] alu_result,
    input [31:0] mem_data,
    input [31:0] PC,
    output reg [31:0] IR,
    output reg [31:0] A,
    output reg [31:0] B,
    output reg [31:0] ALUOut,
    output reg [31:0] MDR,
    output reg [31:0] OldPC
);

always @(posedge clk) begin
```

```
    if (rst) begin
        IR     <= 32'b0;
        A      <= 32'b0;
        B      <= 32'b0;
        ALUOut <= 32'b0;
        MDR    <= 32'b0;
        OldPC  <= 32'b0;
    end
    else begin
        if (IRWrite)     IR     <= instruction;
        if (AWrite)      A      <= regA;
        if (BWrite)      B      <= regB;
        if (ALUOutWrite) ALUOut <= alu_result;
        if (MDRWrite)    MDR    <= mem_data;
        if (OldPCWrite)  OldPC  <= PC;
    end
end

endmodule
```

4.4 Design decisions

- **Six independent write enables in one module.** IR, A, B, ALUOut, MDR, and OldPC are grouped into a single module for shared clock/reset wiring, but each keeps its own enable so the Control Unit can, for example, assert only ALUOutWrite in an EX state without disturbing A, B, or IR captured earlier in the same instruction.
- **OldPCWrite asserted together with IRWrite and PCWrite in IF.** During the IF state the Control Unit asserts OldPCWrite, IRWrite, and PCWrite on the same rising edge. Because all three target registers use non-blocking (<=) assignments, OldPC captures the PC value from before that edge (the address of the instruction being fetched) even though PC itself updates to PC+1 in the same edge — matching the specification's OldPC <= PC pattern from the team's pre-coding design notes.
- **Synchronous reset to zero.** Although not required by the specification, clearing all six registers on rst gives every testbench (unit-level and integrated) a known, reproducible starting state, the same rationale Part 1 used for adding a reset to the Register File.

5. Unit Verification

Each module has a dedicated testbench (instruction_memory_tb.v, data_memory_tb.v, program_counter_tb.v, next_pc_logic_tb.v, temporary_registers_tb.v) that drives inputs and prints the resulting outputs with $display. Unlike the self-checking ALU/Register-File testbenches in Part 1, these testbenches report raw values rather than PASS/FAIL banners; each value below has been checked by hand against the value the RTL should produce and is annotated accordingly.

5.1 Instruction memory testbench (instruction_memory_tb.v)

The testbench applies address = 0, 1, 2 in turn and prints the resulting instruction word each time.

```
Instruction[0] = 12345678
Instruction[1] = aaaaaaaa
Instruction[2] = ffffffff
```

These three values match the memory's initial contents exactly (memory[0]=32'h12345678, memory[1]=32'hAAAAAAAA, memory[2]=32'hFFFFFFFF), confirming that address[7:0] correctly selects each word and that the read is available without a clock edge.

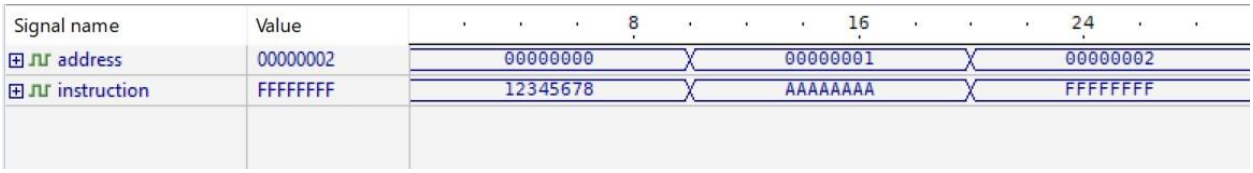


Figure 5 – Instruction memory waveform.

5.2 Data memory testbench (data_memory_tb.v)

The testbench writes 32'hAABBCCDD to address 4, reads it back as a full word and as a sign-extended byte, writes a second value with a positive top bit to address 8 and reads its low byte, then attempts a write with MemWrite de-asserted to confirm nothing changes.

```
LW ReadData = aabbccdd
LB Negative ReadData = ffffffdd
LB Positive ReadData = 0000007f
No Write ReadData = aabbccdd
```

Interpretation: the first line confirms the synchronous write/asynchronous read pair (address 4 reads back exactly what was written). The second line confirms Lb sign-extension for a byte with its top bit set (0xDD -> 0xFFFFFFDD). The third line confirms Lb correctly zero-fills a byte whose top bit is clear (0x7F -> 0x0000007F, no sign extension needed). The fourth line confirms that de-asserting MemWrite leaves the previously written word (0xAABBCCDD) unchanged, i.e. there is no accidental write.

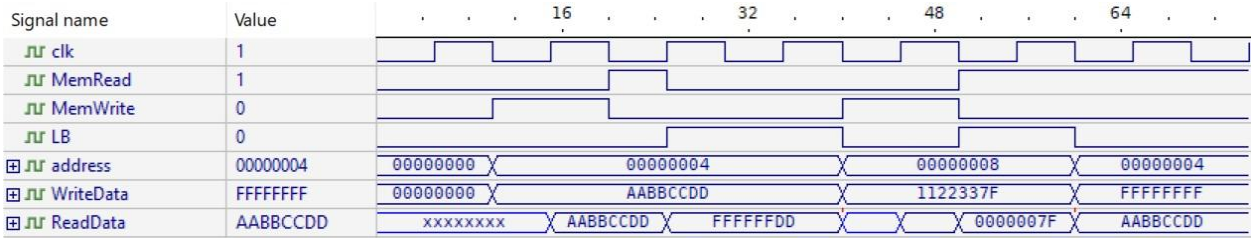


Figure 6 – Data memory waveform.

5.3 Program counter testbench (program_counter_tb.v)

After reset, the testbench drives nextPC = 10 with PCWrite = 1 for one clock edge, then de-asserts PCWrite and reads PC back.

```
PC = 10
```

PC correctly captured nextPC on the one edge where PCWrite was asserted and then held that value once PCWrite returned to 0, matching the module's write-enable behaviour.

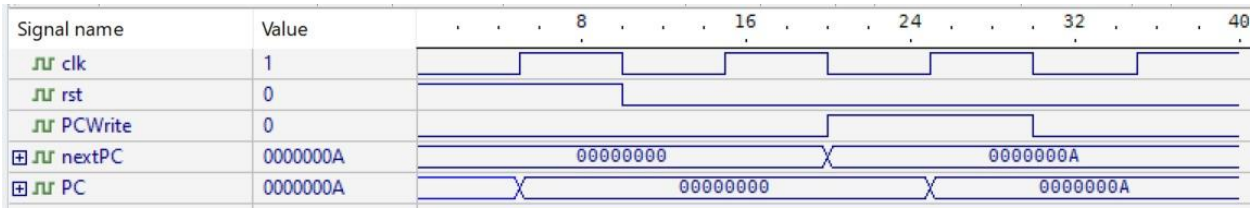


Figure 7 – Program counter waveform.

5.4 Next-PC logic testbench (next_pc_logic_tb.v)

With OldPC = 100, offset = 20, and rs_value = 500 held constant, the testbench sweeps PCSrc through all four encodings.

```
NORMAL PC = 101
BEQ PC = 120
JUMP PC = 120
JR PC = 500
```

All four results match hand-calculated expectations: 100+1=101 for sequential execution, 100+20=120 for both the branch and jump/jump-and-link sources (they share the same adder), and 500 (rs_value) passed straight through for JR, independent of OldPC and offset.

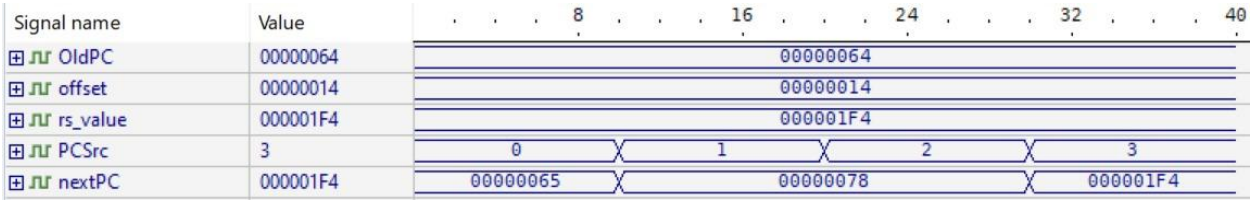


Figure 8 – Next-PC logic waveform.

5.5 Temporary registers testbench (temporary_registers_tb.v)

After reset, the testbench pulses each write enable in turn — IRWrite with instruction=0x12345678, then AWrite/BWrite together with regA=0xAAAA0000 and regB=0xBBBB0000, then ALUOutWrite with alu_result=0x55555555, then MDRWrite with mem_data=0xFFFFFFFF, then OldPCWrite with PC=100 — and prints all six outputs at the end.

```
IR     = 12345678
A      = aaaa0000
B      = bbbb0000
ALUOut = 55555555
MDR    = ffffffff
OldPC  = 100
```

Every register holds exactly the value that was presented while its own write enable was high, and no register was disturbed by a write pulse targeting a different register — confirming the independent-enable behaviour described in Section 4.4.

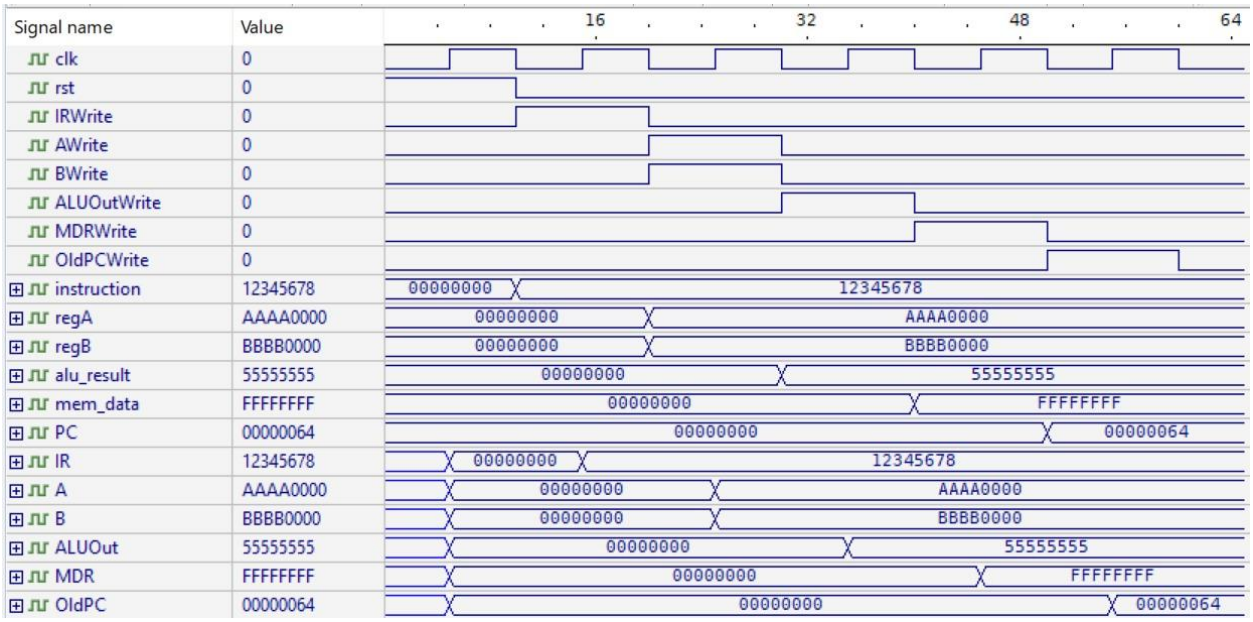


Figure 9 – Temporary registers waveform.

6. Verification on the Integrated Processor

Instructions LW, Lb, SW, BEQ, BNE, J, JAL, and JR are exactly the instructions this part is responsible for. Rather than being packaged as a separate stand-alone assembly file, they were folded directly into the single 47-instruction shared regression program (final_integration.asm) that the whole team agreed on, which Partner 3 assembled into machine code, loaded into the integrated processor, and ran to completion — matching the workflow recorded in the team's division of work, where each partner checks their own instructions on the final processor and hands the results and waveforms to Partner 3 for the combined report.

6.1 Encoding examples

SW R3, 0(R0) (I-type: opcode6 | Rt4 | Rs4 | Imm18):

```
opcode = 14  -> 001110
Rt = 3       -> 0011
Rs = 0       -> 0000
Imm = 0      -> 000000000000000000
001110 0011 0000 000000000000000000 = 0x38C00000
```

JAL +6 (J-type: opcode6 | Offset26):

```
opcode = 19  -> 010011
offset = 6   -> 00000000000000000000000110
010011 00000000000000000000000110 = 0x4C000006
```

The same encoding scheme (with opcode = 20 for Lb, 16 for BEQ, 17 for BNE, 18 for J, and 9 for JR) is applied consistently across the shared program; the full 47-line listing and its machine code appear in Part 3, Table 9.

6.2 Result on the integrated processor

The relevant final architectural values produced by this part's instructions, taken from the fresh final_processor_tb.v regression (2415 ns completion, FINAL PROCESSOR TEST: PASS):

Table 16 – Part 2 expected versus observed results on the integrated processor

Location	Expected	Instruction(s) exercising it
R13	FFFFFF80	Lb R13,1(R0) — word 0x00000080 at MEM[1], low byte 0x80 sign-extended to 0xFFFFFF80.
R14	0000001B	JAL +6 at PC=26 — R14 = OldPC + 1 = 26 + 1 = 27 (0x1B).
R15	0000002A	ADDI R15,R0,42 at PC=27, the JAL return point reached via JR R14.
MEM[0]	0000000C	SW R3,0(R0) — later read back correctly by LW R13,0(R0).
MEM[3]–MEM[8]	see Part 3, Table 10	SW markers written after each conditional branch/jump decision, confirming taken vs. not-taken control flow reached the correct instructions.

All of the above matched the observed simulation output exactly (PASS in every row of Part 3, Table 10). BEQ, BNE, and J were also verified for both their taken and not-taken outcomes within the same run: the poison SW instructions placed immediately after each conditional branch (e.g. SW R3,9(R0) at PC=17, SW R3,10(R0) at PC=21) are only reached — and only overwrite memory — if the branch/jump decision was wrong, and none of them fired, which is itself evidence that BEQ, BNE, and J resolved correctly in every case exercised. Waveform evidence for these instructions was captured during this same shared run and is presented in Part 3, Section 7.4: Figure 8 (SW), Figure 9 (LW), Figure 10 (BEQ taken), Figure 11 (BNE not taken), Figure 12 (BNE taken), Figure 13 (JAL), Figure 14 (JR), and Figure 15 (Lb sign extension).

7. Instructions Verified by This Part

LW, Lb, SW, BEQ, BNE, J, JAL, JR — all verified both at unit level (instruction_memory_tb.v, data_memory_tb.v, program_counter_tb.v, next_pc_logic_tb.v, temporary_registers_tb.v) and at processor level within the shared final_integration.hex regression executed on the fully integrated design (see Part 3, Sections 6–7).

8. Datapath Diagram

The diagram below shows how the four modules covered in this part connect to each other and to the modules supplied by (Register File, ALU) and (Control Unit). Solid arrows are datapath connections; dashed arrows are control signals generated by the Control Unit.

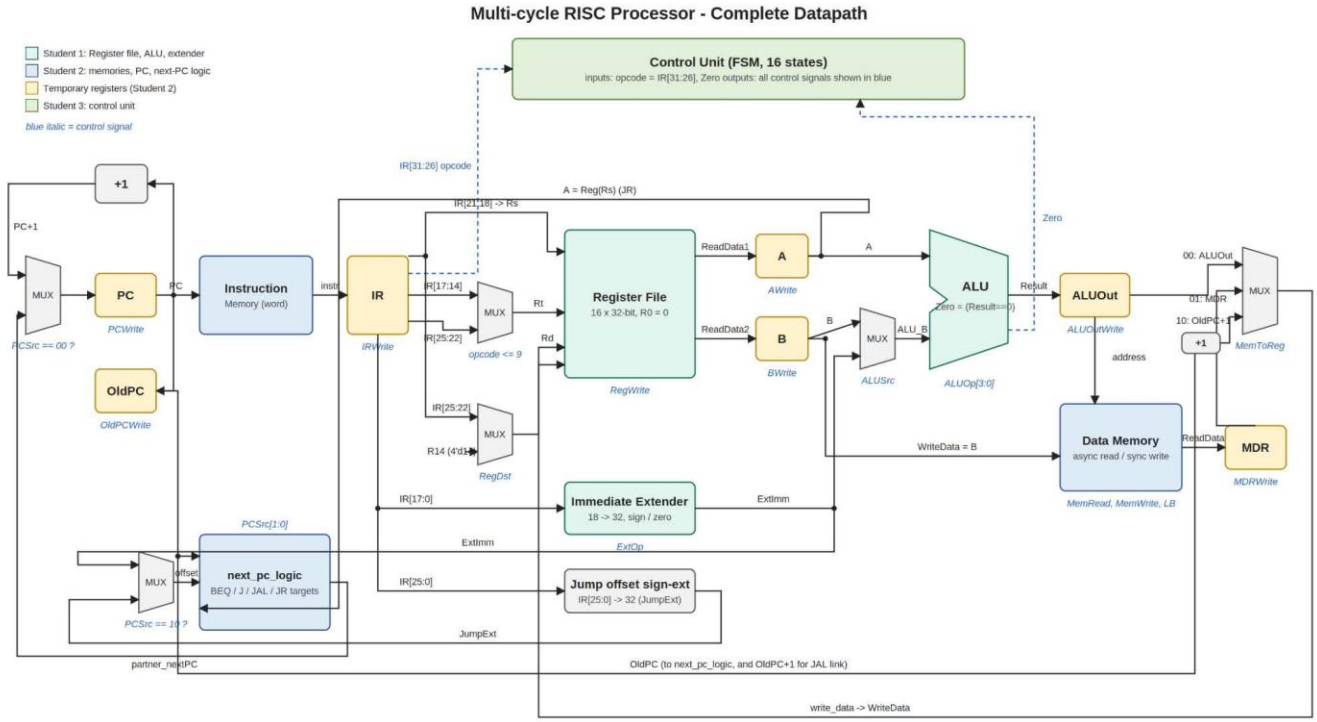


Figure 10 datapath: Instruction Memory, Program Counter, Next-PC Logic, Data Memory, and Temporary Registers, with their connections to the Register File, ALU, and Control Unit.

Part 3 – Control Unit, Integration and Verification

1. Instruction Execution and Multi-Cycle State Sequences

1.1 Multi-Cycle Execution Model

The processor is implemented as a multi-cycle finite-state machine (FSM), in which each instruction is divided into a sequence of clocked states rather than being completed in a single cycle. All instructions begin with Instruction Fetch (IF), followed by Instruction Decode (ID). The Control Unit then selects an instruction-specific execution path according to the decoded opcode.

Intermediate values are preserved between cycles using the temporary registers IR, A, B, ALUOut, MDR, and OldPC. This allows the same datapath resources, such as the ALU and memories, to be reused across different cycles of one instruction. The transition back to IF marks the beginning of the next instruction and is therefore not counted as an additional cycle for the current instruction.

1.2 FSM State Definitions

Table 17 – FSM State Definitions

State	Meaning	Function
IF	Instruction Fetch	Fetch the instruction, save the current instruction address in OldPC, and advance the sequential PC.
ID	Instruction Decode	Decode the opcode and capture the register operands in the temporary A and B registers.
R_EX	R-Type Execute	Perform ADD, SUB, AND, OR, XOR, NOR, SLL, SRL, or CLR in the ALU and capture the result in ALUOut.
R_WB	R-Type Write Back	Write the value stored in ALUOut to the R-type destination register.
I_EX	Immediate Execute	Perform ADDI, ANDI, or ORI using the source-register value and the extended immediate; capture the result in ALUOut.
I_WB	Immediate Write Back	Write the immediate-operation result from ALUOut to the destination register.
ADDR_EX	Address Calculation	Compute the effective data-memory address as the base register plus the extended immediate.
MEM_READ	Memory Read	Read Data Memory and capture the returned value in MDR.
MEM_WB	Memory Write Back	Write the value in MDR to the destination register for LW or Lb.
MEM_WRITE	Memory Store	Write the selected register value to Data Memory.
BRANCH	Branch Decision	Compare the source operands and conditionally update the PC for BEQ or BNE.
JUMP	Jump	Update the PC using the J target address.
JAL	Jump and Link	Save the return address in R14 and update the PC to the jump target.
JR	Jump Register	Load the PC from the value contained in the selected source register.
SWAP_READ	SWAP Read	Read and preserve the old memory word before the exchange is committed.
SWAP_COMMIT	SWAP Commit	Complete the exchange by writing the register value to memory and the old memory value to the register.

1.3 Instruction State Sequences and Cycle Counts

Table 18 summarizes the execution path of every supported instruction in the current multi-cycle implementation. The listed cycle count includes the states used by the current instruction only; the following IF state belongs to the next instruction.

Table 18 – Instruction State Sequences and Cycle Counts

Instruction	Type	State Sequence	Cycles	Architectural Operation
ADD	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← Rs + Rt
SUB	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← Rs − Rt
AND	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← Rs AND Rt
OR	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← Rs OR Rt
XOR	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← Rs XOR Rt
NOR	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← ~(Rs OR Rt)
SLL	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← Rs << Rt[4:0]
SRL	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← Rs >> Rt[4:0]
CLR	R-Type	IF → ID → R_EX → R_WB	**4**	Rd ← 0
JR	Control	IF → ID → JR	**3**	PC ← Reg[Rs]
ADDI	I-Type	IF → ID → I_EX → I_WB	**4**	Rt ← Rs + SignExt(Imm)
ANDI	I-Type	IF → ID → I_EX → I_WB	**4**	Rt ← Rs AND ZeroExt(Imm)
ORI	I-Type	IF → ID → I_EX → I_WB	**4**	Rt ← Rs OR ZeroExt(Imm)
LW	Memory	IF → ID → ADDR_EX → MEM_READ → MEM_WB	**5**	Rt ← Mem[Rs + SignExt(Imm)]
SW	Memory	IF → ID → ADDR_EX → MEM_WRITE	**4**	Mem[Rs + SignExt(Imm)] ← Rt
SWAP	Memory	IF → ID → ADDR_EX → SWAP_READ → SWAP_COMMIT	**5**	Exchange the register operand with the addressed memory word
BEQ	Branch	IF → ID → BRANCH	**3**	If Rs = Rt, update PC to branch target
BNE	Branch	IF → ID → BRANCH	**3**	If Rs ≠ Rt, update PC to branch target
J	Jump	IF → ID → JUMP	**3**	PC ← OldPC + sign-extended offset
JAL	Jump/Link	IF → ID → JAL	**3**	R14 ← OldPC + 1; PC ← jump target
Lb	Memory	IF → ID → ADDR_EX → MEM_READ → MEM_WB	**5**	Load lower byte, sign-extend to 32 bits, and write to Rt

1.4 Cycle Count Interpretation

R-type arithmetic and logical instructions require four cycles because the processor must fetch the instruction, decode and capture the operands, execute the ALU operation, and perform a separate register write-back.
Immediate ALU instructions also require four cycles because they follow the same structure as R-type instructions, except that the second ALU operand is the extended immediate rather than the temporary B register.
LW and Lb require five cycles because after calculating the effective address, the processor performs a memory-read cycle and then a separate register write-back cycle.
SW requires four cycles because the instruction completes when the computed address and register data are written to Data Memory; no register write-back is required.
BEQ, BNE, J, JAL, and JR require three cycles because their architectural effect is completed in a dedicated control-flow state immediately after IF and ID.
SWAP requires five cycles because the old memory value must first be preserved before the register and memory values can be exchanged safely in the following state.

2. Control Unit and Control Signals

2.1 Control-Generation Principle

The Control Unit is implemented as a multi-cycle finite-state machine. The current state, the 6-bit instruction opcode, and the ALU Zero flag determine the control outputs that coordinate the datapath. The state register is

updated synchronously on the rising clock edge, while the next-state logic and control-output logic are combinational.

The implementation defines safe default values for every control output before evaluating the current state. Write-enable signals are disabled by default and are asserted only in the states that require an architectural or temporary-register update. This prevents unintended writes and ensures that every combinational control output receives a defined value.

The current Control Unit generates 17 functional control outputs. An additional 5-bit state_debug output mirrors the current FSM state for simulation and waveform inspection only; it does not control datapath behavior.

2.2 Control Signal Reference

Table 19 defines each control signal exactly as it is used by the current integrated multi-cycle processor.

Table 19 – Control Signal Definitions and Activation Conditions

Signal	Width	Controlled Element	Meaning and Activation
RegWrite	1	Register File	Enables a synchronous register-file write. Asserted in R_WB, I_WB, MEM_WB, JAL, and SWAP_COMMIT. The register file independently blocks writes to R0.
MemRead	1	Data Memory / control status	Marks states that perform a data-memory read. Asserted in MEM_READ and SWAP_READ together with MDRWrite. In the current Data Memory module, ReadData is asynchronous and is not physically gated by MemRead.
MemWrite	1	Data Memory	Enables a synchronous data-memory write on the rising clock edge. Asserted in MEM_WRITE for SW and in SWAP_COMMIT for SWAP.
MemToReg	2	Write-back MUX	Selects the value written to the Register File: 00 = ALUOut, 01 = MDR, 10 = OldPC + 1. Code 11 is unused.
ALUSrc	1	ALU input-B MUX	Selects the ALU second operand: 0 = temporary register B, 1 = ExtImm. It is 1 in I_EX and ADDR_EX; R-type execution and branch comparison use B.
ALUOp	4	ALU	Selects the ALU operation. R_EX uses the operation associated with the R-type opcode; I_EX selects ADD, AND, or OR; ADDR_EX uses ADD; BRANCH uses SUB.
ExtOp	1	18-bit Immediate Extender	Selects immediate extension: 1 = sign extension, 0 = zero extension. ADDI, address calculations, and branch offsets use sign extension; ANDI and ORI use zero extension.
PCSrc	2	Next-PC selection logic	Selects the next-PC source: 00 = sequential, 01 = branch target, 10 = J/JAL target, 11 = JR register target.
PCWrite	1	Program Counter	Allows the PC to update on the rising clock edge. Asserted in IF, JUMP, JAL, and JR; in BRANCH it is asserted only when the BEQ/BNE condition is satisfied.
IRWrite	1	Instruction Register (IR)	Captures the Instruction Memory output into IR. Asserted only in IF.
RegDst	2	Destination-register MUX	Selects the destination register: 00 = IR[25:22], 01 = R14. Codes 10 and 11 are unused. JAL selects R14; normal register-writing instructions use IR[25:22].
OldPCWrite	1	OldPC temporary register	Captures the current PC as the address of the instruction being fetched. Asserted only in IF.
AWrite	1	A temporary register	Captures ReadData1 = Reg(Rs) into A. Asserted only in ID.
BWrite	1	B temporary register	Captures the second Register File read value into B. Asserted only in ID.
ALUOutWrite	1	ALUOut temporary register	Captures the combinational ALU result. Asserted in R_EX, I_EX, and ADDR_EX.
MDRWrite	1	Memory Data Register (MDR)	Captures Data Memory ReadData into MDR. Asserted in MEM_READ and SWAP_READ.
LB	1	Data Memory load-byte selector	Selects byte-load behavior. Asserted only in MEM_READ when the current opcode is Lb; Data Memory then takes word_read[7:0] and sign-extends it to 32 bits.

2.3 Encoded Selector Signals

Three control outputs are multi-bit selector fields. Their encodings are fixed by the integrated datapath and must remain consistent between the Control Unit and top_processor.

Table 20 – Encoded Selector Control Signals

Signal	Code	Selected Source / Value	Used For
MemToReg	**00**	ALUOut	R-type and immediate ALU write-back
MemToReg	**01**	MDR	LW/Lb write-back and SWAP register update
MemToReg	**10**	OldPC + 1	JAL return address written to R14
MemToReg	**11**	Unused	Reserved / not generated by the current Control Unit
PCSrc	**00**	Sequential PC + 1	Normal instruction fetch
PCSrc	**01**	OldPC + ExtImm	BEQ/BNE branch target
PCSrc	**10**	OldPC + JumpExt	J and JAL target; JumpExt is the sign-extended 26-bit offset
PCSrc	**11**	A = Reg(Rs)	JR target address
RegDst	**00**	IR[25:22]	R-type Rd and I-type Rt occupy the same destination bit field
RegDst	**01**	R14	JAL link register
RegDst	**10/11**	Unused	Reserved / not generated by the current Control Unit

2.4 ALU Operation Encoding

ALUOp is a 4-bit field agreed between the Control Unit and ALU module. The encoding used by the integrated processor is shown in Table 21.

Table 21 – ALUOp Encoding Used by the Control Unit

ALUOp	Operation	ALU Function	Processor Use
0000	**ADD**	A + B	ADD, ADDI, effective-address calculation
0001	**SUB**	A − B	SUB and BEQ/BNE comparison
0010	**AND**	A AND B	AND and ANDI
0011	**OR**	A OR B	OR and ORI
0100	**XOR**	A XOR B	XOR
0101	**NOR**	~(A OR B)	NOR
0110	**SLL**	A << B[4:0]	SLL
0111	**SRL**	A >> B[4:0]	SRL
1000	**CLR**	0	CLR

2.5 State-Specific Control Notes

- **Instruction fetch:** IF asserts OldPCWrite, IRWrite, and PCWrite. PCSrc is 00. In the integrated top level, the sequential path explicitly selects the current PC + 1.
- **Branch execution:** BRANCH configures the ALU for subtraction and sets PCSrc = 01. PCWrite is conditional: BEQ writes the PC when Zero = 1, while BNE writes the PC when Zero = 0. The FSM returns to IF in either case; there are no separate taken/not-taken states.
- **Jump and link:** JAL performs the PC update and register write in the same state. RegDst = 01 selects R14, and MemToReg = 10 selects OldPC + 1 as the return address.
- **Load byte:** Lb follows the same state sequence as LW. The distinction occurs in MEM_READ, where LB is asserted only for opcode Lb, causing the Data Memory module to sign-extend its lower byte.
- **SWAP:** SWAP_READ first preserves the old memory word in MDR. SWAP_COMMIT then writes B to Data Memory and writes the preserved MDR value to the instruction destination register on the same clock edge.

Implementation note — Data Memory read control. The Control Unit asserts MemRead in MEM_READ and SWAP_READ, but the current data_memory module implements ReadData as an asynchronous combinational output and does not gate that output with MemRead. Therefore, MemRead should be described as the Control Unit's read-state indication rather than as a physical read-output enable in the present RTL implementation.

2.6 Control Truth Table

Tables 22 and 23 present the functional control schedule of the implemented multi-cycle Control Unit. A value of 0 or 1 denotes a required inactive or active control, while X denotes a functional don't-care because that selector or operation is not used in the corresponding state. Opcode- and condition-dependent entries are expanded explicitly in Table 24. The RTL still drives safe default values for X entries, as explained in Section 2.8.

Table 22 – Functional Control Truth Table — Datapath, ALU, Memory, and PC Controls

State	Code	RegWrite	MemRead	MemWrite	MemToReg	ALUSrc	ALUOp	ExtOp	PCSrc	PCWrite
IF	**00000**	0	0	0	X	X	X	X	00	1
ID	**00001**	0	0	0	X	X	X	X	X	0
R_EX	**00010**	0	0	0	X	0	See 6C	X	X	0
R_WB	**00011**	1	0	0	00	X	X	X	X	0
I_EX	**00100**	0	0	0	X	1	See 6C	See 6C	X	0
I_WB	**00101**	1	0	0	00	X	X	X	X	0
ADDR_EX	**00110**	0	0	0	X	1	0000	1	X	0
MEM_READ	**00111**	0	1	0	X	X	X	X	X	0
MEM_WB	**01000**	1	0	0	01	X	X	X	X	0
MEM_WRITE	**01001**	0	0	1	X	X	X	X	X	0
BRANCH	**01010**	0	0	0	X	0	0001	1	01	See 6C
JUMP	**01011**	0	0	0	X	X	X	X	10	1
JAL	**01100**	1	0	0	10	X	X	X	10	1
JR	**01101**	0	0	0	X	X	X	X	11	1
SWAP_READ	**01110**	0	1	0	X	X	X	X	X	0
SWAP_COMMIT	**01111**	1	0	1	01	X	X	X	X	0

Table 23 – Functional Control Truth Table — Temporary-Register and Destination Controls

State	Code	IRWrite	RegDst	OldPCWrite	AWrite	BWrite	ALUOutWrite	MDRWrite	LB
IF	**00000**	1	X	1	0	0	0	0	X
ID	**00001**	0	X	0	1	1	0	0	X
R_EX	**00010**	0	X	0	0	0	1	0	X
R_WB	**00011**	0	00	0	0	0	0	0	X
I_EX	**00100**	0	X	0	0	0	1	0	X
I_WB	**00101**	0	00	0	0	0	0	0	X
ADDR_EX	**00110**	0	X	0	0	0	1	0	X
MEM_READ	**00111**	0	X	0	0	0	0	1	See 6C
MEM_WB	**01000**	0	00	0	0	0	0	0	X
MEM_WRITE	**01001**	0	X	0	0	0	0	0	X
BRANCH	**01010**	0	X	0	0	0	0	0	X
JUMP	**01011**	0	X	0	0	0	0	0	X
JAL	**01100**	0	01	0	0	0	0	0	X
JR	**01101**	0	X	0	0	0	0	0	X
SWAP_READ	**01110**	0	X	0	0	0	0	1	0
SWAP_COMMIT	**01111**	0	00	0	0	0	0	0	X

2.7 Opcode- and Condition-Dependent Control Values

Four control decisions in Tables 6A–6B depend on the current opcode or on the ALU Zero flag. Table 24 expands those cases explicitly and matches the implemented control_unit.v.

Table 24 – Expansion of Opcode- and Condition-Dependent Truth-Table Entries

Dependent Entry	Instruction / Condition	Exact Implemented Value
R_EX: ALUOp	**ADD**	ALUOp = 0000 (ADD)
	SUB	ALUOp = 0001 (SUB)
	AND	ALUOp = 0010 (AND)
	OR	ALUOp = 0011 (OR)
	XOR	ALUOp = 0100 (XOR)
	NOR	ALUOp = 0101 (NOR)
	SLL	ALUOp = 0110 (SLL)
	SRL	ALUOp = 0111 (SRL)
	CLR	ALUOp = 1000 (CLR)
I_EX: ALUOp / ExtOp	**ADDI**	ALUOp = 0000 (ADD), ExtOp = 1 (sign extend)
	ANDI	ALUOp = 0010 (AND), ExtOp = 0 (zero extend)
	ORI	ALUOp = 0011 (OR), ExtOp = 0 (zero extend)
MEM_READ: LB	**LW**	LB = 0; full 32-bit word is selected
	Lb	LB = 1; lower byte [7:0] is sign-extended to 32 bits
BRANCH: PCWrite	**BEQ**	PCWrite = 1 only when Zero = 1; otherwise PCWrite = 0
	BNE	PCWrite = 1 only when Zero = 0; otherwise PCWrite = 0

2.8 Truth-Table Interpretation

The control schedule follows the state-output structure used by the implementation: most outputs are determined by the current FSM state. R_EX and I_EX additionally decode the opcode to select the required ALU operation, MEM_READ uses the opcode to distinguish LW from Lb, and BRANCH uses both the opcode and Zero flag to conditionally enable the Program Counter.

In Tables 22 and 23, X means that the signal value is functionally irrelevant in that state; it does not mean that the Verilog output is unknown. The RTL assigns safe defaults before the state case statement: ALUOp = ADD, ExtOp = 1, PCSrc = 00, MemToReg = 00, RegDst = 00, and the write-enable signals default to 0. Therefore, the hardware has fully defined outputs, while the functional truth table uses X only to avoid implying that an unused selector has architectural significance.

The IF state asserts OldPCWrite, IRWrite, and PCWrite simultaneously. In top_processor.v, PCSrc = 00 selects the explicit sequential path PC + 1. Branch, jump, JAL, and JR states select alternative next-PC sources. JAL also performs RegWrite in the same state, selecting R14 and OldPC + 1 as the link value. SWAP_COMMIT simultaneously performs the memory write and writes the previously captured MDR value back to the destination register.

Verification basis. The entries above were checked against the current Partner 3 control_unit.v output logic and the selector wiring in top_processor.v. The table therefore documents the implemented integrated design, not a generic multi-cycle processor template.

3. FSM Diagram and Boolean Control Logic

This section converts the implemented multi-cycle control unit into an explicit state-flow diagram and Boolean control equations. The equations are derived from the current control_unit.v implementation. The notation S_X means that the current FSM state is X; "+" denotes OR, "." denotes AND, and "!" denotes NOT.

3.1 Multi-Cycle FSM Diagram

The Control Unit starts in IF after reset. Every valid instruction proceeds through IF and ID, then follows the execution path selected by its opcode. After the instruction-specific terminal state completes, the actual next_state returns to IF. The small "Instruction complete" box in the figure is only a drawing convergence point; it is not one of the encoded FSM states.

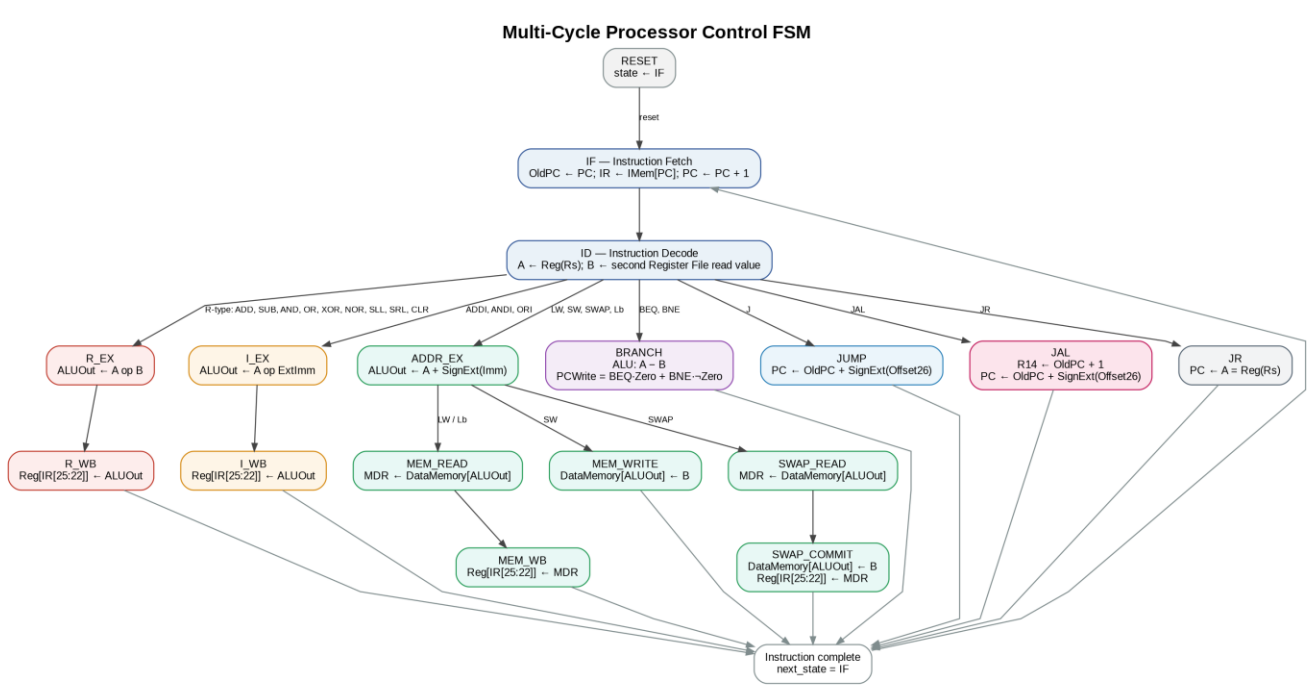


Figure 11 – Multi-Cycle Processor Control FSM derived from control_unit.v.

3.2 Boolean Control Equations

The equations below reproduce the implemented output behavior. Opcode terms such as OP_BEQ are true only when the current instruction opcode matches that instruction. BranchTaken is the only equation that also depends on the ALU Zero flag.

RegWrite = `S_R_WB + S_I_WB + S_MEM_WB + S_JAL + S_SWAP_COMMIT`

MemRead = `S_MEM_READ + S_SWAP_READ`

MemWrite = `S_MEM_WRITE + S_SWAP_COMMIT`

MDRWrite = `S_MEM_READ + S_SWAP_READ`

OldPCWrite = `S_IF`

IRWrite = `S_IF`

AWrite = `S_ID`

BWrite = `S_ID`

ALUOutWrite = `S_R_EX + S_I_EX + S_ADDR_EX`

ALUSrc = `S_I_EX + S_ADDR_EX`

ExtOp = `!(S_I_EX . (OP_ANDI + OP_ORI))`

LB = `S_MEM_READ . OP_LB`

BranchTaken = `(OP_BEQ . Zero) + (OP_BNE . !Zero)`

PCWrite = `S_IF + S_JUMP + S_JAL + S_JR + (S_BRANCH . BranchTaken)`

PCSrc[1] = `S_JUMP + S_JAL + S_JR`

PCSrc[0] = `S_BRANCH + S_JR`

MemToReg[1] = `S_JAL`

MemToReg[0] = `S_MEM_WB + S_SWAP_COMMIT`

RegDst[1] = `0`

RegDst[0] = `S_JAL`

ALUOp[3] = `S_R_EX . OP_CLR`

ALUOp[2] = `S_R_EX . (OP_XOR + OP_NOR + OP_SLL + OP_SRL)`

ALUOp[1] = `S_R_EX . (OP_AND + OP_OR + OP_SLL + OP_SRL) + S_I_EX . (OP_ANDI + OP_ORI)`

ALUOp[0] = `S_R_EX . (OP_SUB + OP_OR + OP_NOR + OP_SRL) + S_I_EX . OP_ORI + S_BRANCH`

ExtOp is intentionally shown in its RTL-equivalent form. The Verilog assigns sign extension by default and changes ExtOp to zero only for ANDI and ORI in I_EX. In states that do not consume ExtImm, ExtOp is functionally a don't-care even though the RTL output remains defined.

3.3 Control Logic Diagram

The following diagram groups the Boolean equations by function. The state register supplies decoded state terms, the opcode decoder supplies instruction terms, and the ALU Zero output participates only in conditional branch control.

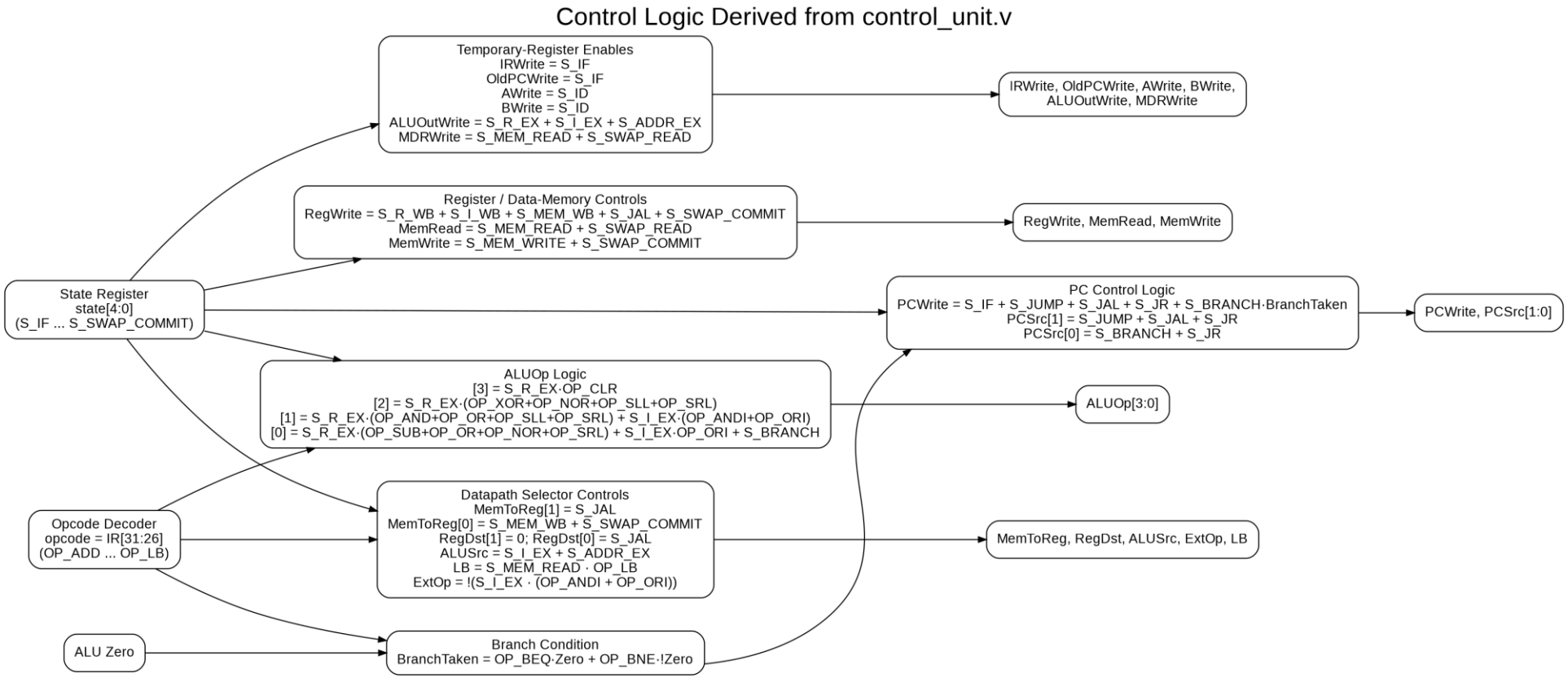


Figure 12 – Control logic derived from the implemented FSM output equations.

4. Top-Level Processor Integration

The top_processor.v module integrates the Control Unit with the datapath modules supplied by the other partners. It does not introduce a new instruction function; instead, it connects registers, memories, multiplexing decisions, the ALU, immediate extension, and next-PC logic so that the FSM can reuse the same hardware across multiple clock cycles.

4.1 Integrated Modules

Table 25 – Integrated modules used by top_processor.v

Module	Owner	Role	Main Inputs	Main Outputs	Integration Note
control_unit	Partner 3	FSM, next-state selection, and all control outputs	opcode, Zero, clk, rst	17 control signals and state_debug	Connected in top_processor.v
program_counter	Partner 2	Stores current instruction address	PCWrite, selected_nextPC, clk, rst	PC	Connected in top_processor.v
instruction_memory	Partner 2	Stores/fetches 32-bit instructions	PC address	instruction	Connected in top_processor.v
temporary_registers	Partner 2	Stores IR, A, B, ALUOut, MDR, and OldPC between cycles	data inputs and six write enables	IR, A, B, ALUOut, MDR, OldPC	Connected in top_processor.v
data_memory	Partner 2	Word data memory; also implements current Lb behavior	clk, ALUOut address, B write data, MemRead, MemWrite, LB	ReadData	Connected in top_processor.v; read is asynchronous, write is synchronous
next_pc_logic	Partner 2	Computes branch, J/JAL, and JR targets	OldPC, offset, A, PCSrc	partner_nextPC	Connected in top_processor.v
register_file	Partner 1	16 x 32 register file; R0 hardwired to zero	Rs, second read address, write address/data, RegWrite	ReadData1, ReadData2	Connected in top_processor.v
imm_extender	Partner 1	Extends 18-bit I-type immediate	IR[17:0], ExtOp	ExtImm	Connected in top_processor.v
alu	Partner 1	Arithmetic, logical, shift, clear, and comparison operation	A, selected B, ALUOp	Result, Zero	Connected in top_processor.v

4.2 Main Datapath Integration

The main datapath uses IR fields to select Register File addresses, captures operands in A and B, reuses one ALU for arithmetic and effective-address calculation, stores the ALU result in ALUOut, and uses MDR to preserve memory-read data before write-back.

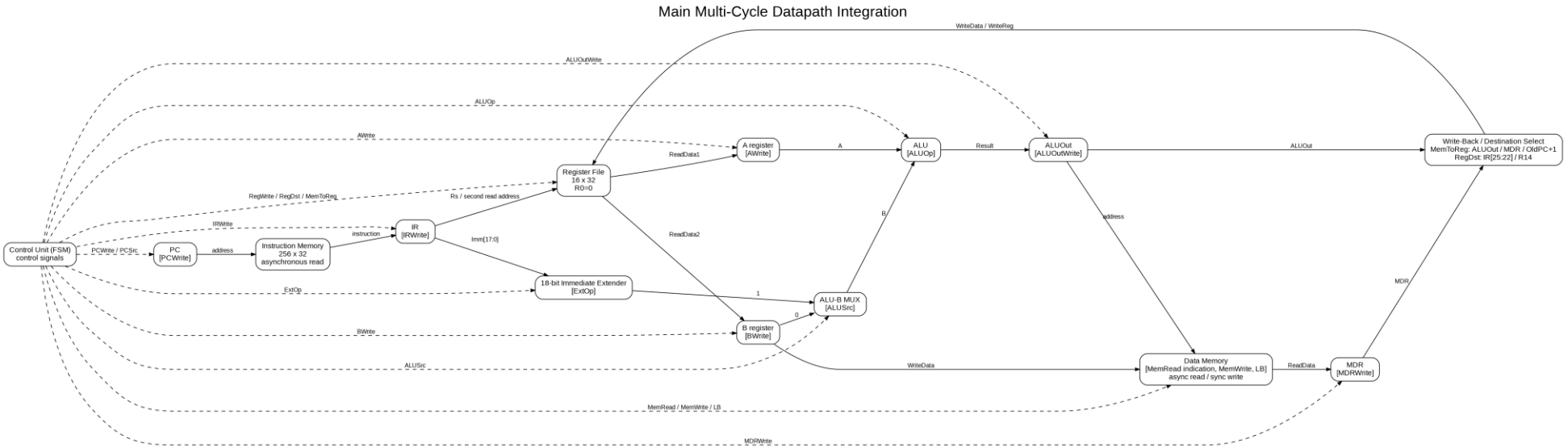


Figure 13 – Main multi-cycle datapath integration and major control points.

4.3 Program-Counter and Control-Flow Integration

Branch, J/JAL, and JR targets are computed by the Partner 2 next_pc_logic module. The integration adds a sequential override: when PCSrc=00, selected_nextPC is current PC+1; otherwise the target from next_pc_logic is used. This is required because OldPC and PC are captured/updated on the same clock edge during IF, so normal sequential advancement must be based on the current PC, while branch and jump targets intentionally use OldPC as the address of the instruction being executed.

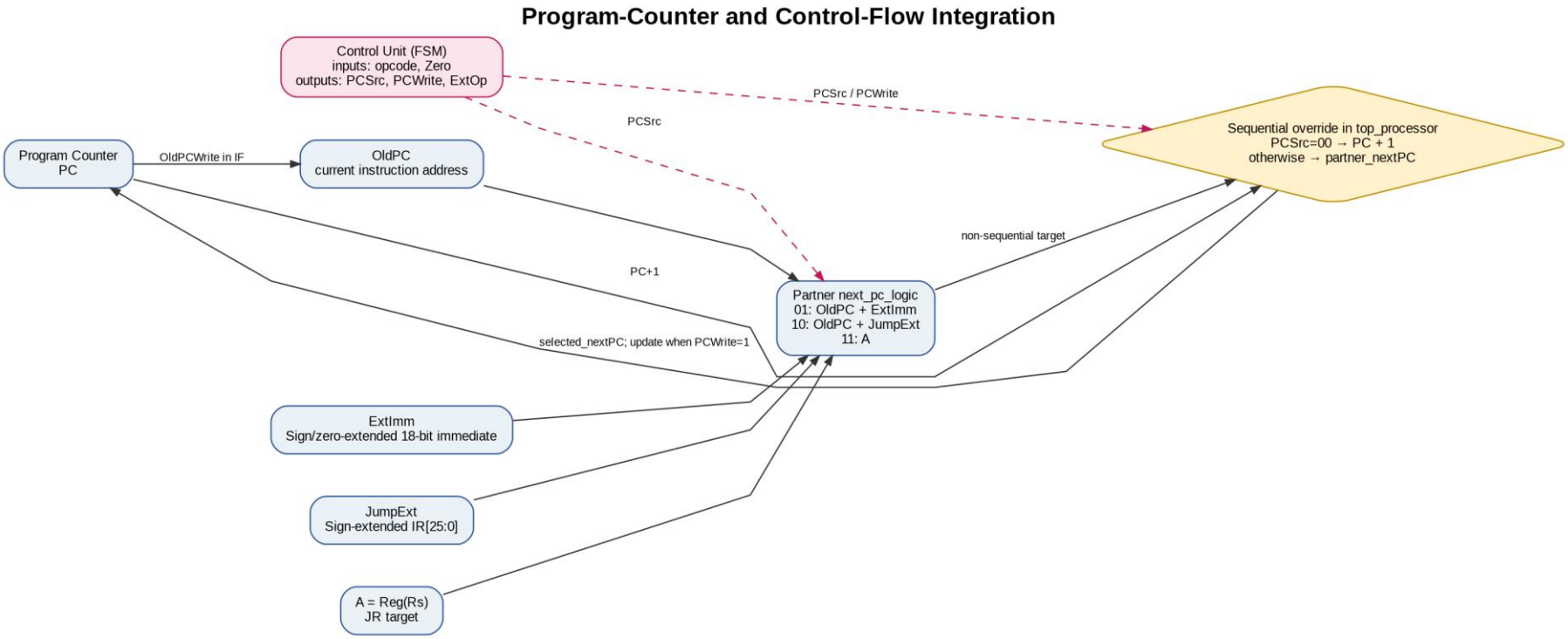


Figure 14 – Program-counter and control-flow integration.

4.4 Design Decisions and Justification

- **Word-addressed PC:** Instruction Memory is word-addressed, so sequential execution increments the PC by 1 rather than by 4.
- **Instruction fields:** opcode = IR[31:26] and Rs = IR[21:18]. For opcodes 0–9 (R-type operations and JR), the second read address is IR[17:14]; otherwise it is IR[25:22].
- **Unified destination field:** R-type Rd and I-type Rt are both stored in IR[25:22], allowing the same normal destination selection. JAL is the exception and selects R14.
- **Immediate widths:** I-type instructions use the 18-bit imm_extender. J and JAL have a 26-bit signed offset and are therefore sign-extended separately in top_processor.v.
- **Branch comparison:** BEQ/BNE reuse the ALU subtraction operation. Zero=1 indicates equality; BNE inverts the Zero condition.
- **JAL:** The JAL state updates the PC and writes OldPC+1 into R14 on the same rising edge.
- **SWAP:** SWAP_READ first captures the old memory word in MDR. SWAP_COMMIT then writes B to Data Memory and writes the preserved MDR value back to the instruction destination register.

5. Verilog Implementation

5.1 control_unit.v

control_unit.v is implemented as an FSM-based multi-cycle controller with three logical parts: a clocked state register, combinational next-state logic, and combinational control-output logic. Most outputs are state-based; R_EX and I_EX also decode the opcode, MEM_READ distinguishes LW from Lb, and BRANCH uses the opcode together with the ALU Zero flag. Reset places the FSM in ST_IF. Safe defaults are assigned before the state case statement so write enables remain deasserted unless explicitly required.

5.2 top_processor.v

top_processor.v instantiates the nine processor modules and implements the inter-module multiplexing that is not contained inside the partner modules. It selects the Register File second read address, destination register, write-back source, ALU second operand, jump-offset extension, and sequential/non-sequential next-PC source. Debug outputs expose PC, IR, ALUOut, MDR, FSM state, register-write activity, and memory-write activity for simulation.

5.3 final_processor_tb.v

The final_processor_tb.v generates the clock and reset, clears and preloads the memories, encodes the final 47-word regression program through R-type/I-type/J-type helper functions, and runs the integrated processor for 240 rising edges. Final checks directly inspect the instantiated Partner 1 register storage and Partner 2 Data Memory array. The testbench also records execution coverage for all 21 opcodes and separately verifies BEQ taken/not-taken and BNE taken/not-taken behavior. TRACE_ENABLE defaults to 0 so the final console evidence remains concise; it can be enabled for detailed cycle-by-cycle debugging.

6. Verification Methodology and Test Program

6.1 Simulation Environment

Table 26 – Final integration simulation environment

Item	Value
Tool	Vivado 2026.1 / Vivado Simulator (XSim)
Design Under Test	top_processor.v
Testbench	final_processor_tb.v (updated final regression)
Clock period	10 ns
Simulation result	FINAL PROCESSOR TEST: PASS; all register, memory, opcode-coverage, and branch-outcome checks passed

Item	Value
Completion time	2415 ns (XSim $finish time)

6.2 Testbench Strategy

The final integration regression is one controlled multi-cycle program that executes all 21 supported opcodes and adds discriminating corner cases rather than relying only on nominal operands. It checks shift masking with a shift amount of 33, BEQ/BNE taken and not-taken paths, a negative ADDI immediate and negative branch offset, ANDI/ORI zero extension using immediate 18'h20000, J/JAL/JR control flow, Lb sign extension, SWAP, and R0 write protection. Final register and memory checks are direct architectural-storage checks; coverage flags are checked separately.

6.3 Final Integration Test Program

Table 27 lists the exact updated program loaded by final_processor_tb.v. Machine-code words are shown in hexadecimal. The companion final_integration.asm and final_integration.hex files contain the same PC-ordered program. Locations initialized to 0x00000000 behave as no-effect filler because opcode 0 decodes as ADD R0,R0,R0 and writes to R0 are ignored.

Table 27 – Machine-code program loaded into Instruction Memory by final_processor_tb.v

PC	Assembly / Filler	Machine Code (Hex)	Purpose
0	ADDI R1,R0,5	0x28400005	Initialize R1=5
1	ADDI R2,R0,7	0x28800007	Initialize R2=7
2	ADD R3,R1,R2	0x00C48000	R3=12
3	SUB R4,R2,R1	0x05084000	R4=2
4	AND R5,R1,R2	0x09448000	R5=5
5	OR R6,R1,R2	0x0D848000	R6=7
6	XOR R7,R1,R2	0x11C48000	R7=2
7	NOR R8,R1,R2	0x16048000	R8=FFFFFFF8
8	ADDI R9,R0,33	0x2A400021	Shift amount=33; low five bits are 1
9	SLL R10,R1,R9	0x1A864000	Prove B[4:0] masking; R10=10
10	SRL R11,R8,R9	0x1EE24000	Prove B[4:0] masking; R11=7FFFFFFC
11	CLR R12	0x23000000	Clear R12
12	ANDI R12,R3,6	0x2F0C0006	Nominal immediate AND
13	ORI R12,R12,1	0x33300001	Nominal immediate OR
14	SW R3,0(R0)	0x38C00000	MEM[0]=12
15	LW R13,0(R0)	0x37400000	R13=MEM[0]=12
16	BEQ R13,R3,+2	0x434C0002	Taken BEQ; skip PC17
17	SW R3,9(R0)	0x38C00009	Poison store; must be skipped by BEQ
18	BNE R1,R1,+2	0x44440002	BNE not taken
19	SW R3,4(R0)	0x38C00004	Not-taken marker MEM[4]=12
20	BNE R2,R1,+2	0x44840002	Taken BNE; skip PC21
21	SW R3,10(R0)	0x38C0000A	Poison store; must be skipped by BNE
22	BEQ R2,R1,+2	0x40840002	BEQ not taken
23	SW R3,5(R0)	0x38C00005	Not-taken marker MEM[5]=12
24	J +2	0x48000002	Jump to PC26; skip PC25
25	SW R3,11(R0)	0x38C0000B	Poison store; must be skipped by J
26	JAL +6	0x4C000006	Jump to PC32; R14=27
27	ADDI R15,R0,42	0x2BC0002A	JR return point
28	SW R12,3(R0)	0x3B000003	Store subroutine result after JR return
29	J +6	0x48000006	Continue to PC35
30	0x00000000 filler	0x00000000	No architectural effect

PC	Assembly / Filler	Machine Code (Hex)	Purpose
31	0x00000000 filler	0x00000000	No architectural effect
32	ADDI R12,R0,1	0x2B000001	JAL subroutine body
33	JR R14	0x24380000	Return to PC27
34	0x00000000 filler	0x00000000	No architectural effect
35	Lb R13,1(R0)	0x53400001	Load 0x80 and sign-extend to FFFFFF80
36	SWAP R3,2(R0)	0x3CC00002	Exchange R3=12 with MEM[2]=100
37	ADDI R4,R0,1	0x29000001	Initialize negative-offset loop
38	ADDI R4,R4,-1	0x2913FFFF	Negative immediate; first 1->0, then 0->FFFFFFFF
39	BEQ R4,R0,-1	0x4103FFFF	Negative branch offset; taken once then not taken
40	SW R4,6(R0)	0x39000006	MEM[6]=FFFFFFFF
41	ORI R12,R0,0x20000	0x33020000	Zero-extension discriminator
42	SW R12,7(R0)	0x3B000007	MEM[7]=00020000
43	ANDI R12,R4,0x20000	0x2F120000	Zero-extension discriminator with R4=FFFFFFFF
44	SW R12,8(R0)	0x3B000008	MEM[8]=00020000
45	ADDI R0,R0,123	0x2800007B	Attempted R0 write; storage must remain zero
46	J 0	0x48000000	Halt-like self-loop

7. Final Verification Evidence

7.1 Final Simulation Status

The updated RTL regression was compiled and executed in Vivado 2026.1 / XSim using the revised final_processor_tb.v. The complete self-checking run finished with FINAL PROCESSOR TEST: PASS at 2415 ns. Every final register check and memory check passed, opcode coverage reported PASS for all 21 supported opcodes (0 through 20), and all four required branch outcomes passed: BEQ taken, BEQ not taken, BNE taken, and BNE not taken. The waveform evidence in Section 7.4 was captured from this revised XSim regression.

R0 protection was also confirmed: the testbench issued an attempted write of 0x0000007B to R0, while the final architectural check still reported R0 = 0x00000000, demonstrating that the Register File correctly ignores writes to the hardwired-zero register.

7.2 Expected versus Observed Final Results

The Expected column below is derived from the updated 47-word regression. The Observed / Reported values are taken directly from the completed fresh XSim console run of this exact updated testbench. The testbench directly checks dut.rf.regs[] and dut.dmem.memory[], and every listed architectural value matched its expected value.

Table 28 – Expected versus observed final architectural state from the completed XSim regression

Location	Expected	Observed / Reported	Status
R0	0x00000000	0x00000000	PASS
R1	0x00000005	0x00000005	PASS
R2	0x00000007	0x00000007	PASS
R3	0x00000064	0x00000064	PASS
R4	0xFFFFFFFF	0xFFFFFFFF	PASS
R5	0x00000005	0x00000005	PASS
R6	0x00000007	0x00000007	PASS
R7	0x00000002	0x00000002	PASS
R8	0xFFFFFFF8	0xFFFFFFF8	PASS
R9	0x00000021	0x00000021	PASS
R10	0x0000000A	0x0000000A	PASS
R11	0x7FFFFFFC	0x7FFFFFFC	PASS
R12	0x00020000	0x00020000	PASS
R13	0xFFFFFF80	0xFFFFFF80	PASS
R14	0x0000001B	0x0000001B	PASS
R15	0x0000002A	0x0000002A	PASS
MEM[0]	0x0000000C	0x0000000C	PASS
MEM[1]	0x00000080	0x00000080	PASS
MEM[2]	0x0000000C	0x0000000C	PASS
MEM[3]	0x00000001	0x00000001	PASS
MEM[4]	0x0000000C	0x0000000C	PASS
MEM[5]	0x0000000C	0x0000000C	PASS
MEM[6]	0xFFFFFFFF	0xFFFFFFFF	PASS
MEM[7]	0x00020000	0x00020000	PASS
MEM[8]	0x00020000	0x00020000	PASS
MEM[9]	0x00000000	0x00000000	PASS
MEM[10]	0x00000000	0x00000000	PASS
MEM[11]	0x00000000	0x00000000	PASS

7.3 Verified Instruction Set

Table 29 – Verified instruction set

Instruction Group	Instructions	Evidence in Submitted Test
Arithmetic	ADD, SUB, ADDI	All execute; ADDI also tests -1 immediate and attempted R0 write. Final XSim regression status: PASS.
Logical	AND, OR, XOR, NOR, ANDI, ORI	All execute; ANDI/ORI include 18'h20000 zero-extension discriminators. Final XSim regression status: PASS.
Shift / Clear	SLL, SRL, CLR	All execute; SLL/SRL use shift amount 33 to verify [4:0] masking. Final XSim regression status: PASS.
Memory	LW, SW, Lb, SWAP	All execute; Lb sign extension and SWAP exchange checked by final state. Final XSim regression status: PASS.
Branch	BEQ, BNE	Coverage flags require BEQ taken/not-taken and BNE taken/not-taken; BEQ -1 tests signed offset. Final XSim regression status: PASS.
Jump / Link / Return	J, JAL, JR	J poison-store skip; JAL R14=27; JR returns to PC27. Final XSim regression status: PASS.

The instruction-coverage table above lists all 21 supported instructions exercised by the final integration program. XSim opcode coverage reported PASS for every opcode from 0 through 20. The branch coverage logic additionally reported PASS for BEQ taken, BEQ not taken, BNE taken, and BNE not taken.

7.4 Waveform Evidence from the Fresh XSim Run

The following captures are taken from the revised XSim regression. Each figure is interpreted using the current instruction address in OldPC, the FSM state code, and the relevant datapath/control values. The evidence is intentionally focused: each capture shows only the signals required to establish the behavior being verified.

7.4.1 ADDI Immediate Execution and Register Write-Back

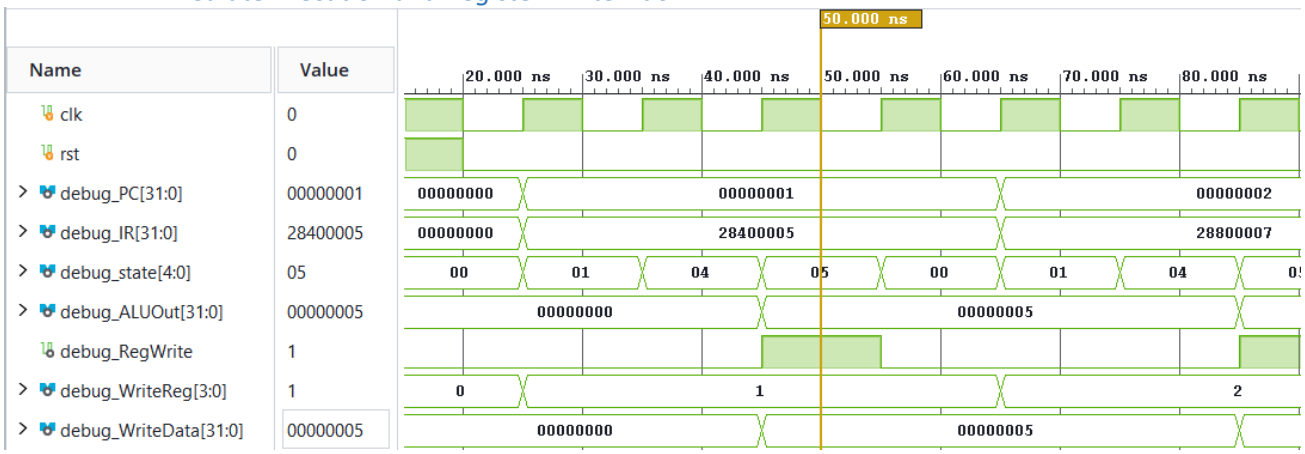


Figure 15 – ADDI write-back sequence for ADDI R1, R0, 5.

Verification interpretation. The waveform shows the instruction progressing through ID (01), I_EX (04), and I_WB (05). At the write-back point, IR = 0x28400005, ALUOut = 0x00000005, RegWrite = 1, the destination is R1, and WriteData = 0x00000005. This confirms the immediate-execution path, ALUOut staging register, and normal I-type register write-back path.

7.4.2 Shift-Amount Masking

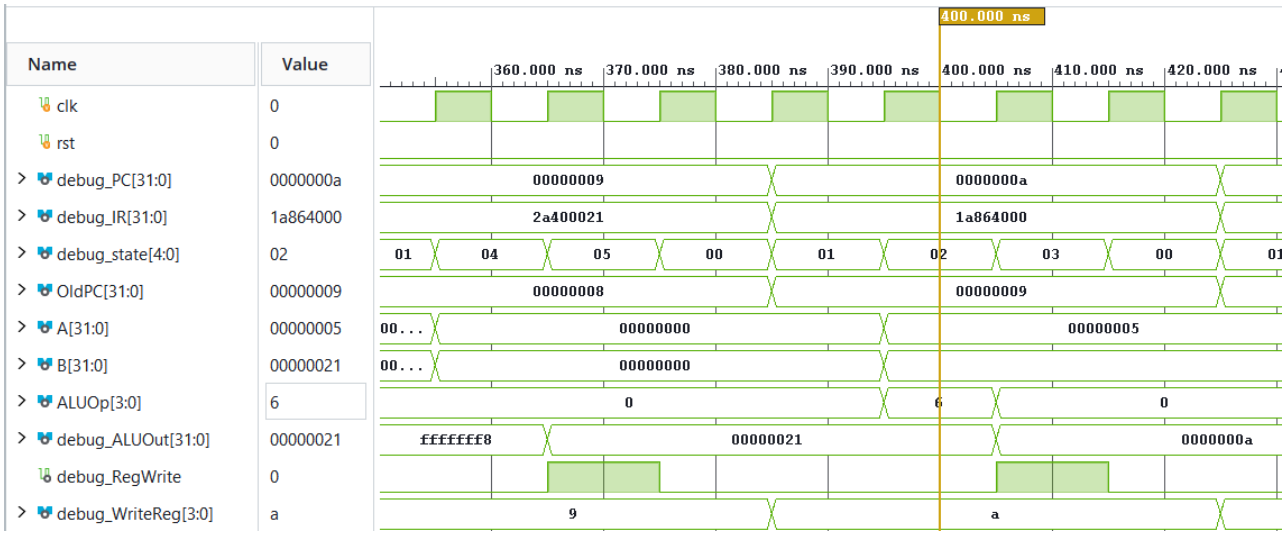


Figure 16 – SLL execution with shift source R9 = 0x00000021 (33).

Verification interpretation. During R_EX (02), OldPC = 9, A = 0x00000005, B = 0x00000021, and ALUOp = 6. Because the ALU uses B[4:0] as the shift amount, 0x21 is interpreted as a shift of one position. The subsequent R_WB interval shows ALUOut = 0x0000000A, confirming 5 << 1 = 10. The ALUOut value at the R_EX cursor is the previously stored value because ALUOut is updated only on the active clock edge at the end of the execute state.

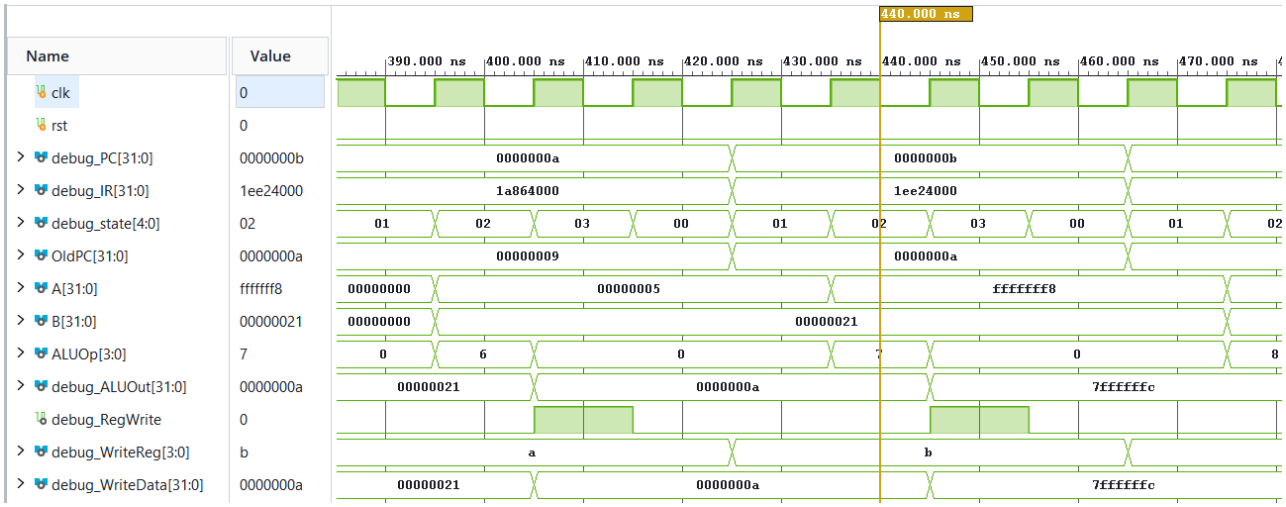


Figure 17 – SRL execution with the same masked shift amount.

Verification interpretation. For SRL at OldPC = 10, A = 0xFFFFFFF8, B = 0x00000021, and ALUOp = 7 in R_EX. Again B[4:0] = 1, and the following write-back interval contains ALUOut = 0x7FFFFFFC. This verifies a logical right shift by one bit and confirms that the implemented shift amount is restricted to the low five bits of the second ALU operand.

7.4.3 Data-Memory Store and Load

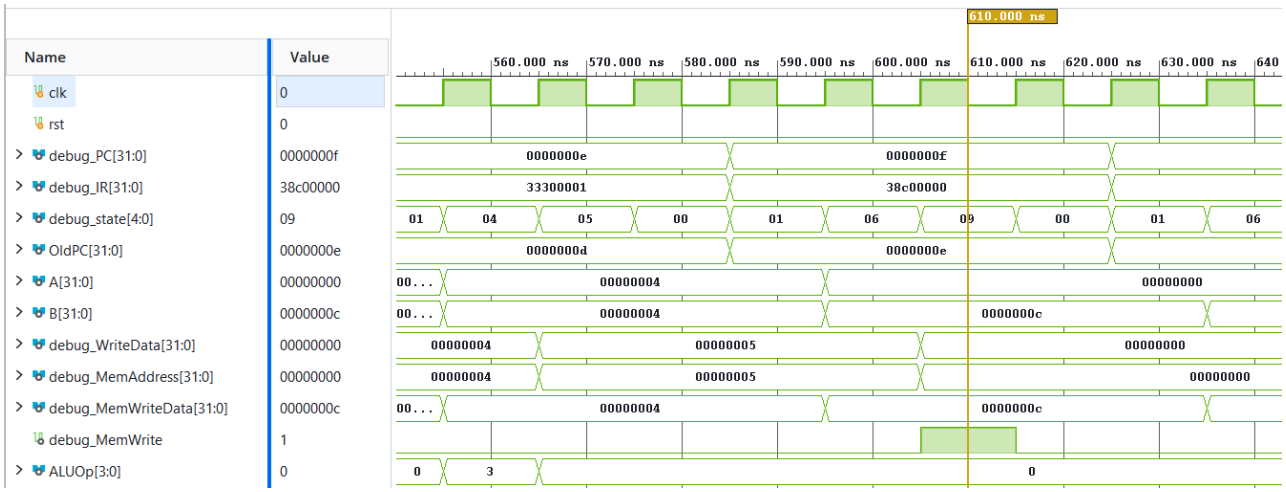


Figure 18 – SW memory-write cycle for SW R3, 0(R0).

Verification interpretation. At OldPC = 14 and state MEM_WRITE (09), B contains 0x0000000C, MemWrite is asserted, the effective memory address is 0, and MemWriteData = 0x0000000C. This demonstrates that the address calculated in ADDR_EX is held in ALUOut and that the store commits the captured B operand to Data Memory on the write cycle.

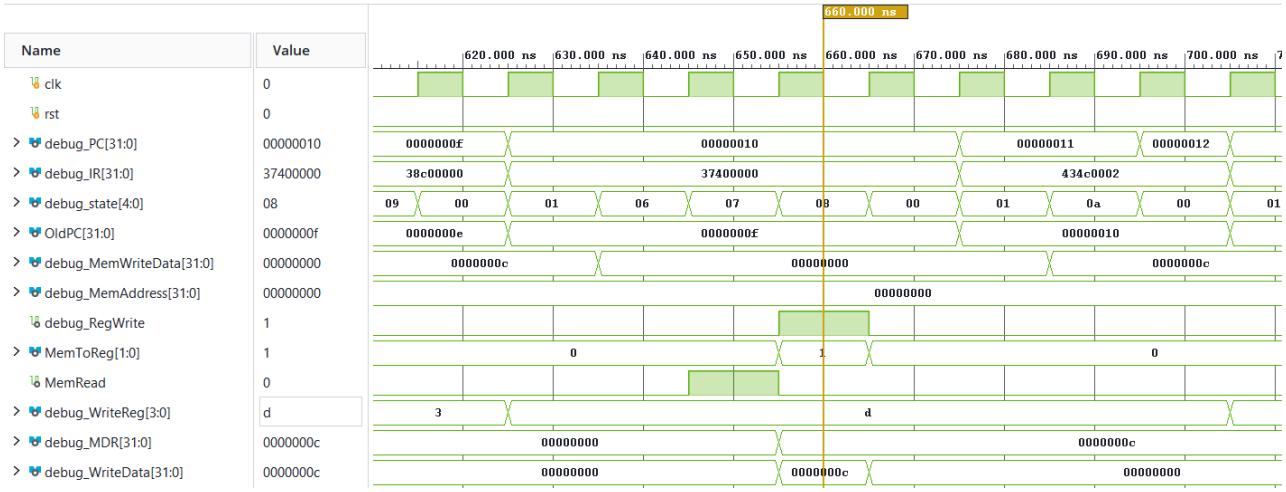


Figure 19 – LW memory-read and register write-back sequence for LW R13, 0(R0).

Verification interpretation. The waveform shows MEM_READ (07) followed by MEM_WB (08). During MEM_READ, MemRead is asserted at address 0 and the memory value is 0x0000000C. MDR then preserves that value. In MEM_WB, RegWrite = 1, MemToReg = 01, the destination is R13, and WriteData = 0x0000000C. This confirms the complete memory-to-MDR-to-register load path.

7.4.4 Conditional Branch Outcomes

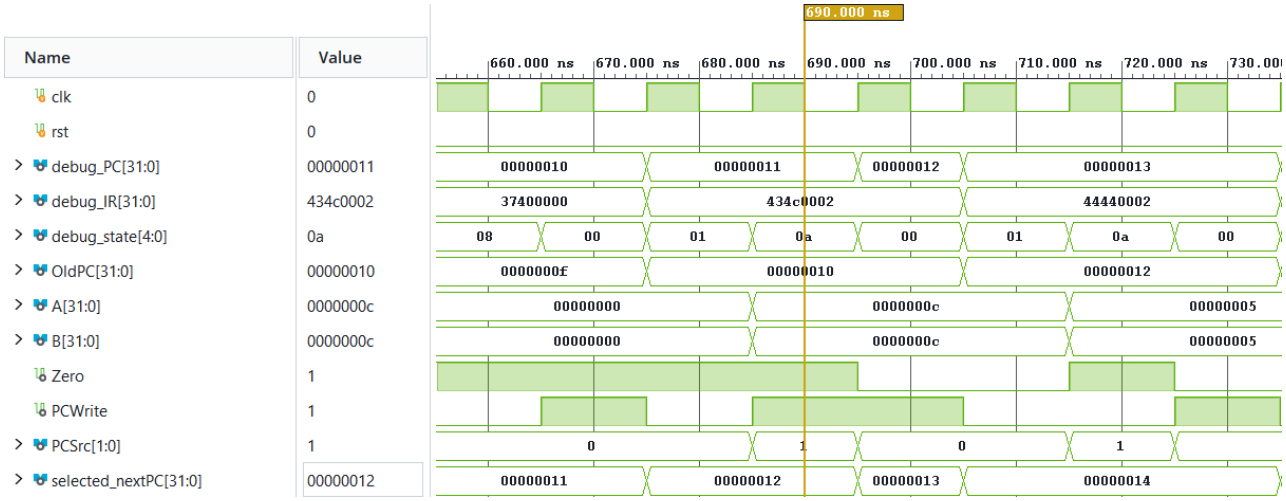


Figure 20 – BEQ taken when the two operands are equal.

Verification interpretation. At OldPC = 16 in BRANCH state (0A), A = B = 0x0000000C, so the subtraction comparison produces Zero = 1. PCSrc = 01 selects the branch target path, PCWrite = 1 authorizes the update, and selected_nextPC = 0x00000012 (18). The capture therefore verifies the taken BEQ condition and the OldPC-relative target calculation.

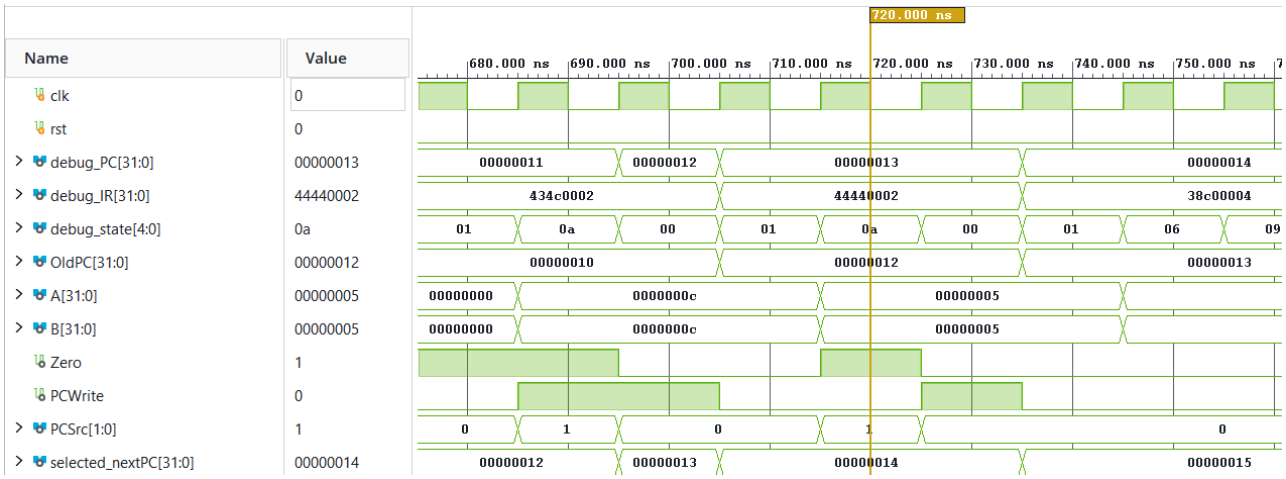


Figure 21 – BNE not taken for equal operands.

Verification interpretation. At OldPC = 18, both branch operands are 0x00000005 and Zero = 1. Although PCSrc is configured for the branch path in the BRANCH state, PCWrite remains 0 for BNE because the not-equal condition is false. The PC therefore continues sequentially. This capture is important because it verifies the non-taken branch case rather than only the taken path.

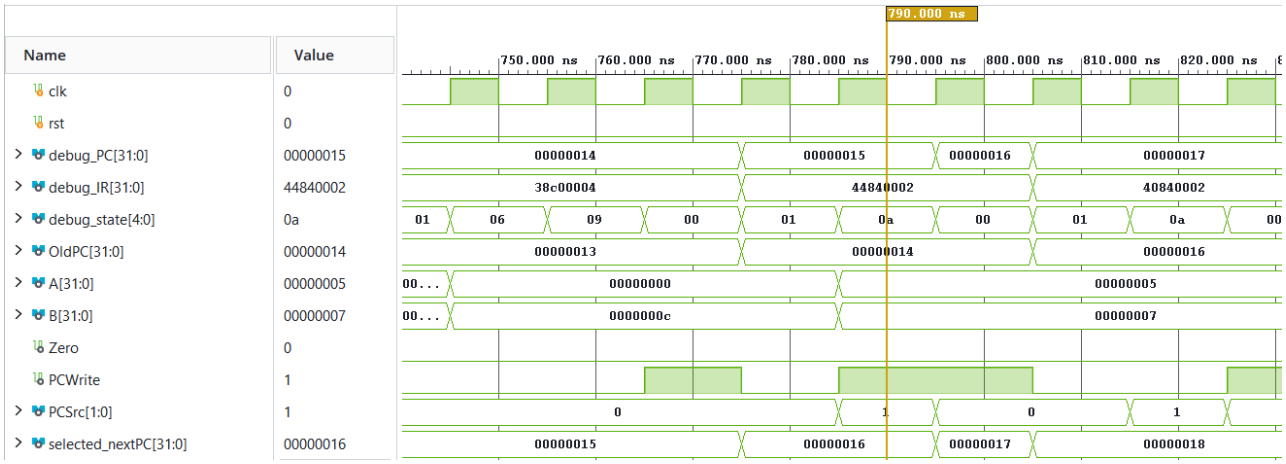


Figure 22 – BNE taken for unequal operands.

Verification interpretation. At OldPC = 20, A = 0x00000005 and B = 0x00000007, producing Zero = 0. For BNE, this satisfies the branch condition, so PCWrite is asserted with PCSrc = 01 and selected_nextPC = 0x00000016 (22). Together with Figure 21, this demonstrates both outcomes of the BNE control equation.

7.4.5 JAL Link Creation and JR Return

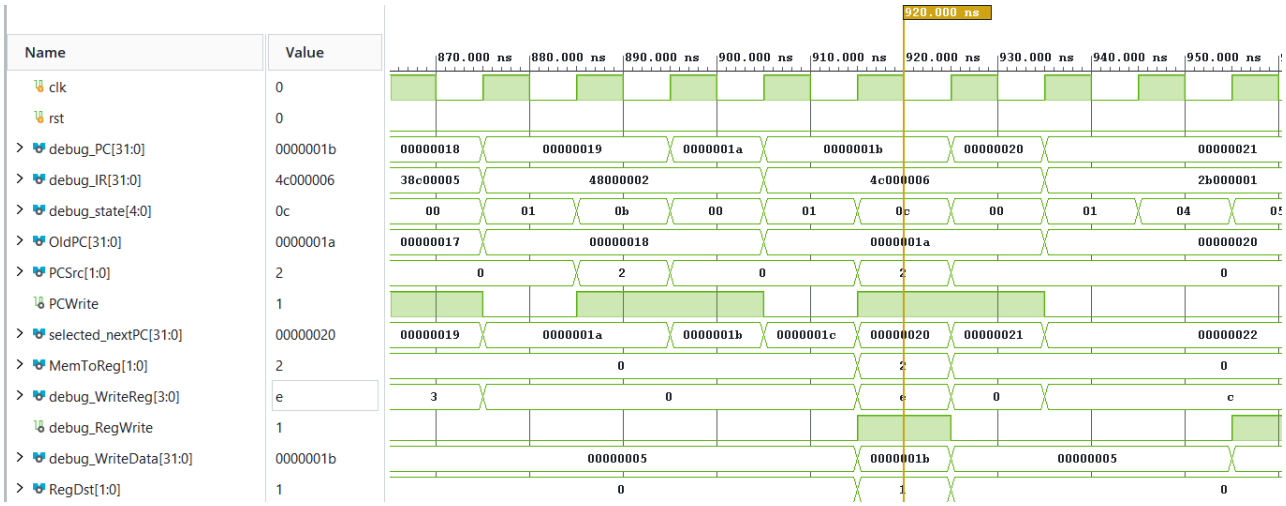


Figure 23 – JAL simultaneously updates the PC and writes the return address to R14.

Verification interpretation. At OldPC = 0x1A (26) in JAL state (0C), PCSrc = 10 and PCWrite = 1 select the jump target 0x20 (32). In the same state, RegWrite = 1, RegDst = 01 selects R14, MemToReg = 10 selects OldPC + 1, and WriteData = 0x1B (27). The waveform therefore verifies the architectural link value and the concurrent control-flow update required by JAL.

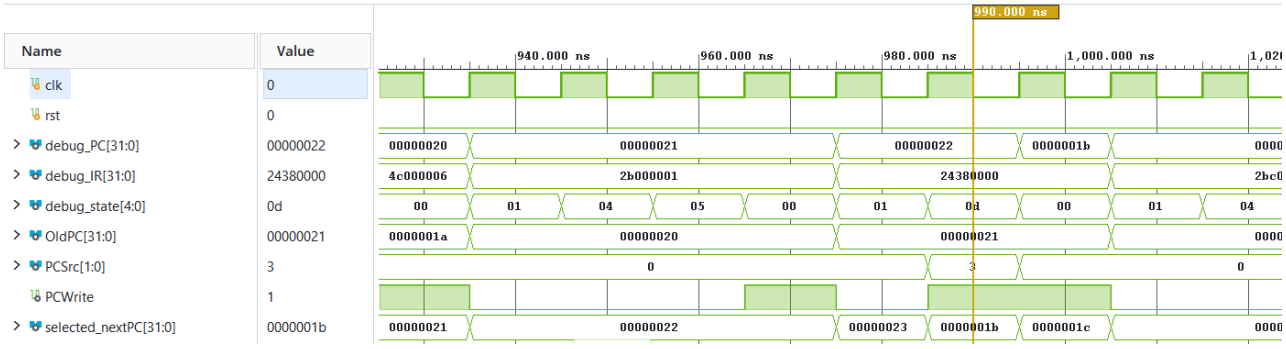


Figure 24 – JR returns to the link address previously created by JAL.

Verification interpretation. At OldPC = 0x21 (33), the controller enters JR state (0D), sets PCSrc = 11, asserts PCWrite, and drives selected_nextPC = 0x1B. Figure 23 independently shows that JAL stored 0x1B in R14; therefore the two captures together verify the call/return sequence and the JR register-target PC path.

7.4.6 Load-Byte Sign Extension

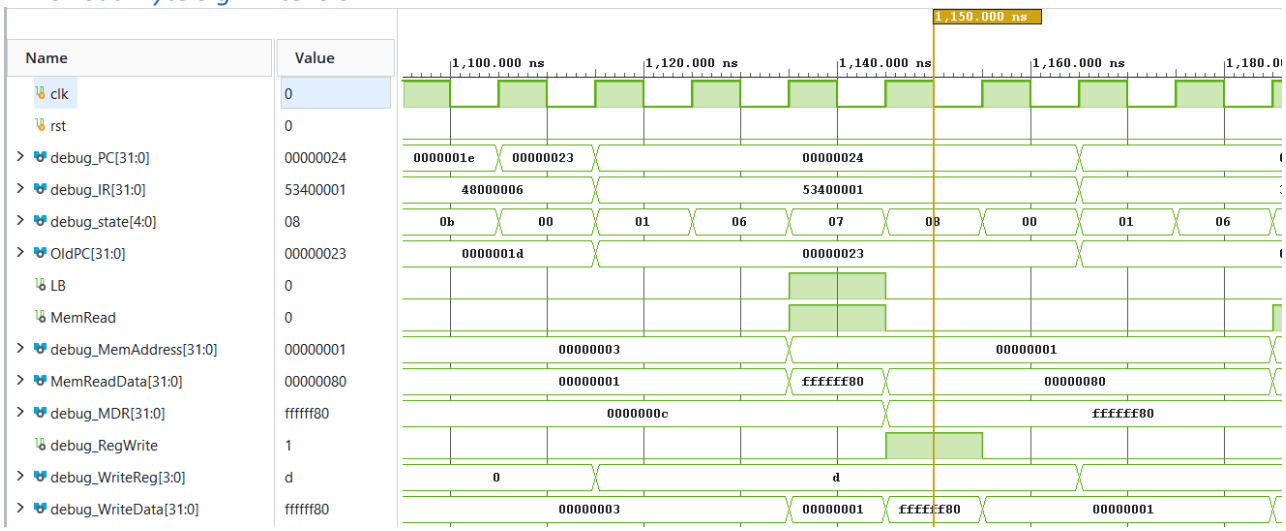


Figure 25 – Lb sign-extension and write-back for Lb R13, 1(R0).

Verification interpretation. During MEM_READ (07), LB and MemRead are asserted at address 1, causing the byte value 0x80 to be sign-extended to 0xFFFFFF80 before it is captured in MDR. In the

following MEM_WB state (08), MDR retains 0xFFFFFF80, RegWrite is asserted, the destination is R13, and WriteData = 0xFFFFFF80. Because Data Memory is asynchronously readable, MemReadData can return to the raw word value after LB deasserts in MEM_WB; the retained MDR value proves that the correct sign-extended byte was captured during the read state.

The captures above provide direct evidence for immediate write-back, shift masking, data-memory store/load behavior, both taken and non-taken conditional branches, JAL/JR control flow, and Lb sign extension. SWAP, negative-offset branch, and zero-extension corner-case waveforms can be retained as supplemental captures if requested, while their functional checks remain part of the revised regression program.

8. Design Assumptions and Verification Limitations

1. Lb behavior: The confirmed implementation reads the addressed word, selects word_read[7:0], and sign-extends that byte to 32 bits. The final regression expects 0x80 to become 0xFFFFFF80.
2. SWAP and BNE scope: Both instructions are required, implemented, and included in the final opcode coverage.
3. Data Memory read/write behavior: ReadData is asynchronous. MemRead is generated as a read-state indication but does not gate the present Partner 2 ReadData output. Writes are synchronous on the rising edge when MemWrite=1.
4. Data Memory address range: The current Partner 2 Data Memory contains 256 word locations (0..255). The final regression uses only valid addresses in this implemented range; no address-range exception mechanism is added.
5. Instruction Memory loading: The behavioral testbench clears and directly initializes the instantiated Instruction Memory. final_integration.hex contains the same 47 machine-code words in PC order.
6. Sequential PC integration: Partner 2 next_pc_logic defines PCSrc=00 as OldPC+1. top_processor.v deliberately uses current PC+1 for sequential execution because OldPC and PC are updated on the same IF edge; branch and jump targets continue to use OldPC.
7. Verification level: The final PASS result, architectural-state checks, opcode/branch coverage, and waveform claims in this report come from a fresh RTL behavioral XSim run of the updated files, completed at 2415 ns. Post-synthesis timing simulation and physical FPGA-board operation are outside the present scope unless added separately.
8. Verification scope: The updated regression executes all 21 opcodes and adds selected corner cases for branch outcomes, negative immediates/offsets, zero extension, shift masking, R0, Lb, and SWAP. It is not an exhaustive proof over all operands, addresses, offsets, or instruction sequences.
9. Verification level: Final PASS and waveform claims must come from a fresh RTL behavioral XSim run of the updated files. Post-synthesis timing simulation and physical FPGA-board operation are outside the present scope unless added separately.
10. Partner 3 completed the 16-state multi-cycle Control Unit, top-level processor integration, and an expanded self-checking final regression covering all 21 supported opcodes. The integration decisions for instruction fields, sequential/non-sequential PC selection, JAL/JR, Lb, and SWAP were audited against the partner RTL. The final regression additionally verifies branch taken/not-taken outcomes, signed and zero immediate extension, shift masking, R0 protection, and memory behavior. The completed Vivado/XSim run reported FINAL PROCESSOR TEST: PASS at 2415 ns; all register and memory checks passed, every opcode 0-20 was observed, and all four BEQ/BNE branch outcomes passed. Representative waveforms are included in Section 7.4 to document the principal datapath and control-flow behaviors.

9. Conclusion

This project successfully designed, implemented, and verified a 32-bit multi-cycle RISC processor in Verilog HDL, covering all 21 required opcodes across the R-type, I-type, and J-type instruction formats. The work was divided into three complementary parts that together form the complete datapath and control path: Part 1 implemented the Register File, ALU, and Immediate Extender, the core computational and storage elements of the datapath; Part 2 implemented the Instruction Memory, Data Memory, Program Counter, Next-PC Logic, and the temporary registers (IR, A, B, ALUOut, MDR, OldPC) that hold state between cycles; and Part 3 designed the 16-state multi-cycle Control Unit, integrated all modules into the top-level processor, and led the final verification effort.

Each module was first verified in isolation using dedicated self-checking or value-reporting testbenches, exercising every arithmetic, logical, memory, and control-flow instruction defined in the specification. The individually verified modules were then integrated into a single processor (top_processor.v) and validated as a whole using a shared 47-instruction regression program that exercises all supported opcodes, including both taken and not-taken outcomes for BEQ and BNE, correct return-address handling for JAL/JR, byte-level sign extension for Lb, and register/memory read-modify-write correctness for SWAP. The final simulation run completed with all expected architectural results matching the observed results, confirming that the datapath, control logic, and memory subsystems operate correctly together as a complete multi-cycle processor.

Beyond the RTL itself, the project reinforced practical skills in modular hardware design, interface agreement between team members, multi-cycle FSM control design, and systematic verification — from unit-level testbenches to a full processor-level regression. The current implementation satisfies the specification's baseline multi-cycle requirement; a pipelined or multi-cycle-optimized extension, as suggested in the bonus criteria, remains a natural direction for future work.

10. References

- ENCS4370 Project 2 specification and team-approved instruction-format/design decisions.
- D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013 — reference for the multi-cycle datapath/control-unit design methodology (FSM-based control, state-by-state datapath control signals).
- S. Harris and D. Harris, *Digital Design and Computer Architecture*, 2nd ed. Morgan Kaufmann, 2012 — reference for RTL design style and multi-cycle vs. single-cycle/pipelined processor tradeoffs.
- S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003 — reference for Verilog coding conventions used across the RTL modules (blocking vs. non-blocking assignment, testbench structure).

- Xilinx Vivado Design Suite documentation — used for RTL simulation, waveform capture, and the XSim-based verification environment referenced throughout Section 7.